

AL-AZHARI LIBRARY

BACKEND IMPLEMENTATION PLAN

Production Implementation Specification

Node.js + Express + TypeScript + MongoDB + Mongoose

Field	Value
Version	1.0.0
Status	Implementation-Ready Specification
Date	23 September 2026
Prepared for	Backend, frontend, QA, operations, and the business owner
Architecture	Modular monolith
Primary locale	Arabic / RTL; English / LTR supported
Database	MongoDB Atlas
Hosting	Hostinger Node.js hosting

Implementation rule: The PRD is the functional authority, the Master Product Brief is the business authority, the IA/Sitemap and UX/UI Specification define routes and interaction contracts, the Design System defines visual constraints, and the MongoDB Implementation Plan defines the baseline data model. Open decisions remain configurable and are not silently converted into business rules.

Table of Contents

1. [Executive Summary](#)
2. [Source Documents & Authority](#)
3. [Backend Goals](#)
4. [Backend Non-Goals](#)
5. [System Architecture](#)
6. [Technology Stack](#)
7. [Architectural Principles](#)
8. [Project Structure](#)
9. [Module Boundaries](#)
10. [Database Architecture](#)
11. [MongoDB Collections](#)
12. [Mongoose Model Strategy](#)
13. [Relationships & References](#)
14. [Indexing Strategy](#)
15. [Transactions & Concurrency](#)
16. [Authentication Architecture](#)
17. [Authorization & RBAC](#)
18. [User & Account Management](#)
19. [Guest Checkout](#)
20. [Cart Architecture](#)
21. [Product & Category APIs](#)
22. [Inventory Architecture](#)
23. [Checkout Architecture](#)
24. [Order Management](#)
25. [Payment Architecture](#)
26. [Payment Proof Workflow](#)
27. [Shipping Architecture](#)
28. [Coupon Architecture](#)
29. [Student Services Architecture](#)
30. [Quotations](#)
31. [Preorders](#)
32. [Returns](#)
33. [Refunds](#)
34. [Notifications](#)
35. [Socket.IO Architecture](#)
36. [Email Architecture](#)
37. [Cloudinary Architecture](#)
38. [WhatsApp Integration](#)
39. [Outbox Architecture](#)
40. [Background Jobs](#)
41. [Cron Jobs](#)

42. [Search & Filtering](#)
 43. [Reports & Analytics](#)
 44. [Audit Logging](#)
 45. [Validation](#)
 46. [Error Handling](#)
 47. [API Response Standards](#)
 48. [API Endpoint Specification](#)
 49. [OpenAPI Documentation](#)
 50. [Security Architecture](#)
 51. [Rate Limiting](#)
 52. [CORS](#)
 53. [Logging & Observability](#)
 54. [Environment Configuration](#)
 55. [Testing Strategy](#)
 56. [Deployment Architecture](#)
 57. [Hostinger Deployment](#)
 58. [Database Deployment](#)
 59. [Backup & Recovery Considerations](#)
 60. [Performance Considerations](#)
 61. [Implementation Phases](#)
 62. [Definition of Done](#)
 63. [Acceptance Criteria](#)
 64. [Risk Register](#)
 65. [Open / Not Specified Items](#)
 66. [Final Technical Decisions](#)
-

1. Executive Summary

AL-AZHARI LIBRARY is a real physical library and store in Qena, Egypt, extending its existing books, educational materials, supplies, gifts, and student-service operation into a structured online channel.

The backend is a modular monolith exposing a versioned REST API at `/api/v1`. MongoDB Atlas is the persisted source of truth. The system supports:

- A books-first product catalog with variants, availability, prices, pre-orders, search, and category browsing.
- Guest and registered-customer carts and checkout.
- Delivery or free pickup, with an estimated shipping cost and an Admin-controlled carrier/final cost.
- Manual payment workflows with screenshots stored through Cloudinary and reviewed by Admin.
- Separate quotation-based student services; services never enter the product cart.
- Customer accounts, order/service history, notifications, returns, and manually recorded refunds.
- Admin/Owner operational workflows protected by permissions and auditable state transitions.
- Socket.IO delivery, email delivery, and an in-process MongoDB outbox that remain secondary to committed database state.

The plan intentionally does not introduce an automated payment gateway, service-file upload, phone OTP, map pins, reviews, ratings, LMS behavior, marketplace behavior, Redis, BullMQ, or Atlas Search for the initial implementation.

1.1 Backend shape

```

flowchart LR
    Angular[Angular client] --> API[Express REST API /api/v1]
    API --> Modules[Modular monolith modules]
    Modules --> Mongo[(MongoDB Atlas)]
    Modules --> Cloudinary[Cloudinary media]
    Modules --> SMTP[SMTP / Gmail / private mail server]
    Modules --> Socket[Socket.IO]
    Modules --> Outbox[MongoDB outboxEvents]
    Outbox --> Worker[Internal worker]
    Worker --> SMTP
    Worker --> Socket

```

1.2 Non-negotiable source-of-truth boundaries

1. MongoDB records, not WhatsApp, are the authoritative order and service records.
2. Order status, payment status, and fulfillment status are separate fields.
3. Product order and service request are separate aggregates with separate lifecycles.
4. Payment is manual. A screenshot is evidence for Admin review, not an automated confirmation.
5. Socket.IO and email are delivery mechanisms. A failed delivery never rolls back a committed business operation.
6. Open policy decisions are represented as configuration or blocked behavior until the owner resolves them.

2. Source Documents & Authority

2.1 Reviewed source set

Source	Role
AL-AZHARI-LIBRARY-PRD_1790189797290.pdf	Functional requirements, business rules, state machines, acceptance criteria, open decisions
AL-AZHARI-LIBRARY-MASTER-PRODUCT-BRIEF_1790189797288.pdf	Business identity, current operation, customer context, product strategy, source-of-truth business constraints
AL-AZHARI-LIBRARY-Information-Architecture-and-Sitemap_1790189797286.pdf	Route architecture, domain boundaries, page inventory, navigation and state destinations
AL-AZHARI-LIBRARY-MongoDB-Implementation-Plan_1790189797289.pdf	Collection inventory, schema intent, references, indexes, money/date conventions, transactions, lifecycle
DESIGN-SYSTEM_1790189797291.md	Locale, RTL/LTR, content and accessibility constraints relevant to API contracts
UX-UI-SPECIFICATION_1790189797292.md	UI data contracts, route behavior, lifecycle presentation, Admin actions, empty/error states
Pasted--You-are-a-Senior-Backend-Architect-MongoDB-Architect-a_1790189809017.txt	Task-specific locked backend architecture, required sections, examples, testing, deployment, and output requirements

2.2 Authority order

1. Explicit functional decisions in the PRD.
2. Business facts and boundaries in the Master Product Brief.
3. IA/Sitemap route and domain structure.
4. MongoDB Implementation Plan for the baseline data model.
5. UX/UI Specification for data needed by screens.
6. Design System for locale, direction, accessibility, and presentation constraints.
7. This plan for implementation mechanisms that do not create new business policy.

2.3 Conflict register

Conflict	Resolution
Payment methods: PRD lists COD plus InstaPay, Vodafone Cash, Orange Cash, Etisalat Cash, and WE Pay; Master Brief confirms COD, cash at library, InstaPay, and Vodafone Cash.	Preserve as OD-20. Store payment methods in settings; do not hard-code the production list until Owner approval.
Shipping configuration: MongoDB plan keeps shipping configuration in settings and lists no shippingRules; the locked backend brief requires a separate shippingRules collection.	Implement shippingRules as the approved backend-plan extension for governorate/zone rules. Keep policy flags and provider-independent settings in settings; snapshot the applied rule on the order.
MongoDB plan says official driver through a data-access layer is acceptable while the locked architecture specifies Mongoose.	Use Mongoose with explicit sessions, transactions, validators, and repository boundaries. No model may bypass the transaction contract.
Some PRD items are labelled product recommendation rather than confirmed requirement.	Implement only where needed for a safe flow, and keep the behavior configurable or documented as a recommendation.

2.4 Status vocabulary used in this plan

- **CONFIRMED:** Must be implemented.
- **TECHNICAL DECISION:** Implementation mechanism required by this plan and not a new business rule.
- **CONFIGURABLE:** Data or policy must be changeable without code deployment.
- **OPEN / NOT SPECIFIED:** Do not invent behavior; block or keep neutral until defined.

3. Backend Goals

1. Expose every confirmed public, customer, service, and Admin workflow through stable /api/v1 contracts.
2. Preserve the library's books-first catalog and separate student-service model.
3. Make product price, availability, order status, payment state, and fulfillment state queryable without staff chat.
4. Prevent overselling with reservation-time atomic checks and final deduction only at delivery/pickup.
5. Keep manual payment verification safe, auditable, and privacy-preserving.
6. Support guest checkout without losing a secure way to view the resulting order.
7. Make lifecycle notifications reliable without making realtime transport authoritative.
8. Keep open decisions configurable: payment methods, shipping rules, return policy, coupon governance, service fields, and verification gating.

9. Provide a production foundation that can run on Hostinger Node.js hosting with MongoDB Atlas.
10. Give developers concrete module boundaries, schemas, API contracts, workflows, tests, and deployment steps.

4. Backend Non-Goals

The backend must not implement or imply:

- An automated payment gateway or direct payment confirmation.
- A service-attachment upload area or `serviceAttachments` collection.
- Phone OTP, Google Maps location pins, or a second checkout for services.
- Reviews, ratings, social feeds, an LMS, course delivery, or marketplace/multi-seller behavior.
- A permanent season-only catalog or seasonal taxonomy mutation.
- Unverified “best,” “number one,” guaranteed availability, or unapproved delivery-time promises.
- A separate `productVariants` collection in MVP; variants are embedded in products.
- Redis/BullMQ as an MVP dependency.
- Atlas Search, Elasticsearch, or another dedicated search engine in the initial implementation.
- Automatic refund execution. Admin records a manual refund.
- WhatsApp as an order, service, payment, or notification system of record.

5. System Architecture

5.1 Deployment topology

```

flowchart TB
    Browser["Browser / Angular"] --> HTTPS["HTTPS same-origin"]
    HTTPS --> Hostinger["Hostinger Node.js process"]
    Hostinger --> Express["Express app"]
    Express --> REST["REST controllers"]
    REST --> Domain["Domain services"]
    Domain --> Repos["Mongoose repositories"]
    Repos --> Atlas["MongoDB Atlas replica set"]
    Domain --> Media["Cloudinary adapter"]
    Domain --> Email["Email adapter"]
    Domain --> WA["WhatsApp URL adapter"]
    Domain --> RT["Socket.IO gateway"]
    Domain --> Outbox["outboxEvents"]
    Outbox --> Worker["Same-process worker"]
  
```

5.2 Request flow

1. Express establishes request ID, security headers, CORS, body limits, and structured logging.
2. Authentication middleware optionally verifies a JWT access token.
3. Authorization middleware checks a permission and, when applicable, record ownership.
4. Controller parses request input with Zod.
5. Application service performs the use case; it does not trust client totals, status, role, or ownership.
6. Repository reads/writes through Mongoose, passing a session for transactional operations.

7. State-changing operations append audit and outbox records in the same transaction.
8. Controller returns the standard response envelope.
9. The internal worker delivers email/socket work after commit; failed work is retried.

5.3 Layer rules

Layer	Responsibility	Must not do
Route	Map HTTP method/path to controller and middleware	Contain business rules
Controller	Parse input, call service, map result to response	Query MongoDB directly
Application service	Orchestrate a use case and transaction	Render UI or send raw HTTP responses
Domain policy	State transitions, money, eligibility, coupon, inventory rules	Read request globals
Repository	Mongoose queries and persistence	Decide who may perform an action
Integration adapter	Cloudinary, email, WhatsApp URL, Socket.IO	Mutate authoritative business records
Worker/job	Retry outbox and scheduled work	Bypass idempotency or authorization rules

6. Technology Stack

Concern	Locked choice
Runtime	Node.js
Language	TypeScript with strict mode
HTTP framework	Express.js
API	REST, versioned under /api/v1
ODM	Mongoose
Database	MongoDB Atlas
Authentication	JWT access token plus refresh token
Refresh storage	Hashed refresh/session record in MongoDB; HttpOnly/Secure cookie for refresh
Password hashing	Argon2id
Validation	Zod
Logging	Pino
Security	Helmet, rate limiting, strict CORS, secure defaults
Media	Cloudinary, with metadata in MongoDB
Realtime	Socket.IO authenticated by access token
Background work	MongoDB Outbox Pattern and an internal worker
Scheduling	Node cron
Email	SMTP/Gmail/private mail server behind an adapter

Concern	Locked choice
API documentation	OpenAPI/Swagger and Postman collection
Tests	Jest, Supertest, MongoDB Memory Server
Frontend consumer	Angular
Deployment	Hostinger Node.js hosting

7. Architectural Principles

1. **Database authority:** only committed MongoDB state is authoritative.
2. **Explicit transitions:** status changes pass through transition maps and domain services.
3. **Immutable financial snapshots:** order items, customer contact, address, price, shipping estimate, and payment method are snapshotted.
4. **Money as integers:** store EGP in integer minor units; never persist floating-point money.
5. **UTC persistence:** store BSON Dates in UTC; localize to Africa/Cairo in the client.
6. **Least privilege:** permission plus record ownership is required.
7. **No invented policy:** missing fields, periods, thresholds, rates, and deadlines remain configurable or OPEN / NOT SPECIFIED.
8. **Short transactions:** perform only authoritative multi-document work inside a transaction.
9. **Idempotent mutations:** retrying a checkout, proof submission, or refund completion must not duplicate the record.
10. **Separate domains:** product commerce, student services, payments, inventory, and notifications have explicit boundaries.
11. **Delivery is secondary:** email, socket, and WhatsApp actions never replace persisted records.
12. **Arabic-first data readiness:** localized fields use { ar, en? }; API routes, identifiers, and codes remain LTR.
13. **Deactivation over deletion:** preserve business history; deactivate catalog/configuration records.
14. **Observable failure:** errors have stable codes, request IDs, structured logs, and recovery guidance.

8. Project Structure

```
src/
├── config/
│   ├── env.ts
│   ├── database.ts
│   └── openapi.ts
├── database/
│   ├── migrations/
│   ├── seed/
│   └── mongoose.ts
├── common/
│   ├── constants/
│   ├── errors/
│   ├── middleware/
│   ├── types/
│   ├── utils/
│   └── http/
├── modules/
│   ├── auth/
│   ├── users/
│   ├── addresses/
│   ├── products/
│   ├── categories/
│   ├── carts/
│   ├── orders/
│   ├── payments/
│   ├── inventory/
│   ├── services/
│   ├── quotations/
│   ├── preorders/
│   ├── returns/
│   ├── refunds/
│   ├── notifications/
│   ├── coupons/
│   ├── content/
│   ├── shipping/
│   ├── reports/
│   ├── audit/
│   └── settings/
├── integrations/
│   ├── cloudinary/
│   ├── email/
│   └── whatsapp/
├── realtime/
│   └── socket/
├── jobs/
│   ├── worker.ts
│   ├── outbox-worker.ts
│   └── cron.ts
├── app.ts
└── server.ts
```

Each module normally contains controllers/, services/, repositories/, models/, schemas/, routes/, types/, and index.ts. A small module may omit an empty folder, but cross-module imports must go through its public application service or repository interface.

9. Module Boundaries

Module	Owns	Publishes/consumes
auth	Users' credentials, sessions, verification, reset tokens	user_registered, email_verified, password_reset
users	Profile and account state	Customer identity snapshots

Module	Owns	Publishes/consumes
addresses	Saved customer addresses	Checkout address snapshot
products	Product identity, localized content, variants, publication	Catalog queries, inventory keys
categories	Stable product taxonomy	Books-first browsing
carts	Guest/user persisted cart	Checkout input
orders	Product-order aggregate and order lifecycle	Payment, inventory, notification, report events
payments	Payment record and payment state	Proof review, refund linkage
inventory	Stock counters and immutable ledger	Reservation, release, deduction
services	Service categories and requests	Quotation and service notifications
quotations	Versioned service quotations	Payment availability after acceptance
preorders	Unavailable-product demand	Availability and fulfillment handoff
returns	Customer return requests	Refund initiation
refunds	Manual refund records	Return/order completion
notifications	Persisted in-app notification records	Socket/email outbox work
coupons	Definitions, validation, redemptions	Order total calculation
content	Homepage/seasonal modules	Public home response
shipping	shippingRules, estimate, provider/final-cost workflow	Order fulfillment snapshot
reports	Read-only aggregations	Admin reports
audit	Immutable critical-operation records	Security/operations review
settings	Versioned operational configuration	Payment, contact, policy, feature flags

No module may write another module's authoritative state by editing its model directly. Cross-module mutations are application-service calls inside one transaction.

10. Database Architecture

10.1 Conventions

- MongoDB Atlas must run as a replica set/tier that supports multi-document transactions.
- Internal relations use `ObjectId`.
- Public references use stable strings such as `ORD-20260923-0001`, `SRV-...`, `PRE-...`, and `RET-...`.
- Localized text uses `{ ar: string; en?: string }`.
- `amountMinor` is a non-negative integer in EGP piastres; currency is snapshotted as "EGP".
- All dates are BSON UTC Dates.
- Every collection has `createdAt` and `updatedAt`; lifecycle records also store domain timestamps.
- API DTOs reject unknown money formats, negative quantities, invalid `ObjectIds`, and unsupported enum values.

10.2 Persistence boundaries

Data	Persistence choice
Product variants	Embedded in products
Order items	Embedded immutable snapshots in orders
Customer/address in order	Embedded snapshot
Status history	Bounded embedded timeline plus external auditLogs
Payment proof files	Referenced paymentProofs documents; Cloudinary metadata only
Inventory ledger	Referenced append-only inventoryTransactions
Notifications	Referenced documents; never an unbounded user array
Service fields	Embedded submitted values plus service-category/form snapshot
Audit logs	Referenced immutable collection
Outbox work	Referenced outboxEvents collection

10.3 No hard delete for business history

Orders, payments, proofs, returns, refunds, audit records, and inventory transactions are never hard-deleted through normal Admin UI. Products, categories, services, coupons, users, content modules, and rules are deactivated or archived.

11. MongoDB Collections

11.1 Collection inventory

Collection	Required	Primary purpose
users	Yes	Customer/Admin/Owner identity
sessions	Yes	Refresh/session records and revocation
authTokens	Yes	Hashed verification/reset tokens with TTL
addresses	Yes	Saved customer addresses
roles	Yes	Extensible permission assignments
categories	Yes	Stable books-first taxonomy
products	Yes	Product, variants, content, availability, stock
carts	Yes	Guest/user persisted cart
orders	Yes	Authoritative product transaction
payments	Yes	Payment state for order or accepted quote
paymentProofs	Yes	Screenshot submissions and review history

Collection	Required	Primary purpose
inventoryReservations	Yes	Reserved stock ownership
inventoryTransactions	Yes	Immutable stock ledger
serviceCategories	Yes	Configurable service catalog/forms
serviceRequests	Yes	Service lifecycle
quotations	Yes	Versioned service quote and decision
preOrders	Yes	Unavailable-product demand
returnRequests	Yes	Item-level return workflow
refunds	Yes	Manual refund records
notifications	Yes	Persisted lifecycle notifications
coupons	Yes	Discount definitions
couponRedemptions	Recommended	Immutable use records
contentModules	Yes	Home and seasonal merchandising
shippingRules	Yes	Governorate/zone/provider-independent estimate rules
settings	Recommended	Operational flags and unresolved policy configuration
auditLogs	Yes	Critical mutation history
outboxEvents	Recommended	Reliable post-commit delivery

11.2 Collection contracts

users

role, name, email, phone, passwordHash, emailVerifiedAt, status, lastLoginAt, refreshTokenVersion, createdAt, updatedAt.

- role: customer | admin | owner.
- Email is lowercased, unique, and valid.
- Phone is canonicalized and unique.
- Password hash is Argon2id output and never returned.
- status: active | suspended.

sessions

userId, tokenHash, userAgent, ipHash, createdAt, lastUsedAt, expiresAt, revokedAt, revokeReason, sessionVersion.

- Refresh token raw values never enter MongoDB.
- expiresAt has a TTL index.
- Logout revokes one session; global logout increments the user's refreshTokenVersion and revokes all active sessions.

authTokens

userId, type, tokenHash, expiresAt, consumedAt, createdAt.

- type: email_verification | password_reset.

- TTL index on expiresAt.
- Only a secure hash is stored.

addresses

userId, label, recipientName, recipientPhone, governorate, city, area, street, buildingNumber, floor, apartment, landmark, notes, isDefault, timestamps.

No map coordinates or location-pin fields exist.

roles

key, displayName, permissionKeys[], isSystem, active, timestamps. Initial roles are owner, admin, and customer; future specialized roles are supported by permission keys without requiring them in the first UI.

categories

slug, name { ar, en? }, parentId, kind: "product", displayOrder, isActive, isMvpEnabled, isBooksCore, timestamps.

Services use serviceCategories. Future product categories can exist inactive but cannot be purchasable in MVP.

products

slug, name, description, categoryId, images[], author, grade, stage, subject, publisher, isbn, educationType, searchText, hasVariants, priceMinor, currency, availability, preOrderEligible, isPublished, returnPolicyFlags, variants[], timestamps.

Each variant has variantId, attributes, label, priceMinor, currency, availability, stockTotal, stockReserved, preOrderEligible, sku, images[], and updatedAt.

carts

ownerType: guest | user, userId, sessionId, items[], currency, expiresAt, version, timestamps.

Each item stores productId, variantId, quantity, display-only unitPriceMinor, product/image snapshots, and addedAt. Services are rejected at the cart API boundary.

orders

reference, customerId, guestAccessTokenHash, customerSnapshot, items[], totals, fulfillment, paymentMethodKey, paymentId, status, statusHistory[], couponSnapshot, submittedAt, acceptedAt, completedAt, cancelledAt, idempotencyKey, version, timestamps.

fulfillment contains method: delivery | pickup, addressSnapshot, provider, shippingStatus, estimateSource, shippingRuleSnapshot, finalCostConfirmedAt.

payments

ownerType: order | serviceQuotation, ownerId, customerId, methodKey, methodSnapshot, amountDueMinor, currency, status, proofRequired, proofSubmissionCount, confirmedAt, rejectedAt, metadata, timestamps.

Payment status is not_submitted | proof_uploaded | under_review | confirmed | rejected | new_proof_requested.

paymentProofs

paymentId, ownerType, ownerId, customerId, submissionNumber, files[], status, customerNote, reviewNote, reviewedBy, reviewedAt, timestamps.

Each file stores clouidinaryPublicId, resourceType, format, bytes, width, height, optional sha256, and no permanent public URL.

inventoryReservations

orderId, orderItemId, productId, variantId, quantity, status: active | released | consumed, createdAt, releasedAt, consumedAt, releasedReason.

Partial unique index prevents two active reservations for the same order item.

inventoryTransactions

Append-only fields: productId, variantId, type: manual_adjustment | reservation | release | deduction | correction, quantityDelta, before, after, sourceType, sourceId, actorId, reason, createdAt.

serviceCategories

slug, name, description, kind, isActive, formVersion, fields[], communicationChannels[], pricingMode: quotation, turnaroundText, codAllowed, timestamps.

Until approved, service field definitions, turnaround text, and COD policy remain empty or null. No invented required fields may be seeded.

serviceRequests

reference, customerId, customerSnapshot, serviceCategoryId, serviceCategorySnapshot, submittedFields, description, status, quotationId, paymentId, communicationContext, statusHistory[], closedReason, timestamps.

Status: submitted | admin_review | quotation_sent | awaiting_payment | payment_verification | payment_confirmed | processing | completed | closed_not_proceeding | closed_declined.

No attachment field is permitted.

quotations

serviceRequestId, customerId, version, amountMinor, currency, status: draft | sent | accepted | rejected, customerDecisionAt, decisionNote, sentBy, acceptedAt, rejectedAt, paymentId, timestamps.

Payment creation is gated on accepted.

preOrders

reference, customerId, customerSnapshot, productId, variantId, productSnapshot, quantity, capturedPriceMinor, currency, status, expectedAvailabilityAt, paymentId, linkedOrderId, allocationSequence, timestamps.

Status: requested | admin_review | accepted | payment_pending | payment_verification | confirmed | available | rejected | fulfilled | cancelled.

No allocation sequence is assigned until the owner resolves the allocation rule.

returnRequests

reference, orderId, customerId, items[], status, customerNote, adminNote, reviewedBy, reviewedAt, refundId, timestamps.

Items contain `orderId`, `quantity`, `reason`, `eligible`, and evidence metadata where the approved policy permits it.

refunds

`orderId`, `returnRequestId`, `customerId`, `amountMinor`, `currency`, `methodKey`, `status`: `initiated` | `completed` | `failed`, `recordedBy`, `recordedAt`, `completedAt`, `note`, `attemptReference`, `timestamps`.

`attemptReference` is an internal reconciliation value; it is not a customer transaction-ID input.

notifications

`recipientUserId`, `recipientRoleContext`, `type`, `title`, `body`, `entityType`, `entityId`, `actionUrl`, `readAt`, `channels`, `deliveryStatus`, `dedupeKey`, `timestamps`.

coupons

`codeNormalized`, `discountType`: `percentage` | `fixed`, `value`, `currency`, `scopeType`: `order` | `product` | `category`, `scopeIds[]`, `active`, `startsAt`, `endsAt`, `usageCount`, `usageLimit`, `minimumOrderMinor`, `stackable`, `customerRestriction`, `version`, `timestamps`.

couponRedemptions

`couponId`, `codeSnapshot`, `orderId`, `customerId`, `discountMinor`, `timestamps`. Unique on (`couponId`, `orderId`).

contentModules

`key`, `title`, `body`, `moduleType`, `productIds[]`, `categoryIds[]`, `startsAt`, `endsAt`, `displayOrder`, `active`, `updatedBy`, `timestamps`. It cannot mutate permanent taxonomy.

shippingRules

`key`, `scopeType`: `governorate` | `city` | `area` | `default`, `scopeValue`, `method`: `delivery` | `pickup`, `estimateMinor`, `currency`, `providerHint`, `serviceable`, `priority`, `active`, `effectiveFrom`, `effectiveTo`, `version`, `updatedBy`, `timestamps`.

The rule is an estimate source, not a customer-selected carrier. If no approved matching rule exists, the API returns a configuration/serviceability error rather than inventing a price.

settings

`key`, `value`, `version`, `updatedBy`, `timestamps`. Initial keys include `payment.methods`, `payment.details`, `contact.whatsapp`, `contact.phone`, `shipping.policy`, `return.policy`, `coupon.policy`, `service.policy`, and `auth.verificationPolicy`. Secrets stay in environment configuration.

auditLogs

`actorId`, `actorRole`, `action`, `entityType`, `entityId`, `previousState`, `newState`, `metadata`, `requestId`, `ipHash`, `createdAt`.

No raw proof URLs, passwords, tokens, or unnecessary PII.

outboxEvents

`eventType`, `aggregateType`, `aggregateId`, `payload`, `dedupeKey`, `status`: `pending` | `processing` | `sent` | `failed`, `attempts`, `availableAt`, `processedAt`, `lastError`, `timestamps`.

12. Mongoose Model Strategy

12.1 Schema rules

- Use `strict`: "throw" for API-owned documents where unknown fields could create security or policy drift.
- Use `timestamps`: `true`.
- Use `select`: `false` on password hashes, token hashes, guest access hashes, and sensitive metadata.
- Use schema enums for all state values, but enforce transitions in services.
- Use custom validators for non-negative integer money and quantities.
- Use `lean()` for read-only list/report queries.
- Pass `{ session }` through every repository method participating in a transaction.
- Do not use `populate()` in financial workflows as a substitute for snapshots.

12.2 Example money type

```
export interface MoneySnapshot {
  amountMinor: number; // integer piastres
  currency: "EGP";
}

export function assertMoney(value: number): void {
  if (!Number.isInteger(value) || value < 0) {
    throw new AppError("VALIDATION_ERROR", "Money must be a non-negative integer", 400);
  }
}
```

12.3 Example Mongoose schema

```
const VariantSchema = new Schema(
  {
    variantId: { type: String, required: true },
    attributes: { type: Map, of: String, required: true },
    label: { type: LocalizedTextSchema, required: true },
    priceMinor: { type: Number, min: 0, required: true },
    currency: { type: String, enum: ["EGP"], required: true },
    availability: {
      type: String,
      enum: ["in_stock", "out_of_stock", "pre_order_eligible"],
      required: true,
    },
    stockTotal: { type: Number, min: 0, required: true, default: 0 },
    stockReserved: { type: Number, min: 0, required: true, default: 0 },
    preOrderEligible: { type: Boolean, required: true, default: false },
  },
  { _id: false, timestamps: { createdAt: false, updatedAt: true } },
);

ProductSchema.index({ categoryId: 1, isPublished: 1, displayOrder: 1 });
ProductSchema.index({ availability: 1, isPublished: 1 });
ProductSchema.index({ isbn: 1 }, { unique: true, sparse: true });
```

12.4 Repository transaction contract

```
export interface RepositoryContext {
  session?: ClientSession;
  requestId: string;
}

export async function withTransaction<T>(  
  work: (session: ClientSession) => Promise<T>,  
) : Promise<T> {  
  const session = await mongoose.startSession();  
  try {  
    let result!: T;  
    await session.withTransaction(async () => {  
      result = await work(session);  
    });  
    return result;  
  } finally {  
    await session.endSession();  
  }  
}
```

The transaction helper must not be used for ordinary reads or single-document content edits.

13. Relationships & References

```
erDiagram
  USERS ||--o{ ADDRESSES : owns
  USERS ||--o{ SESSIONS : has
  USERS ||--o{ CARTS : owns
  USERS ||--o{ ORDERS : places
  USERS ||--o{ SERVICE_REQUESTS : submits
  USERS ||--o{ NOTIFICATIONS : receives
  PRODUCTS }o--|| CATEGORIES : belongs_to
  CARTS ||--o{ CART_ITEMS : embeds
  ORDERS ||--o{ ORDER_ITEMS : embeds
  ORDERS ||--|| PAYMENTS : has
  PAYMENTS ||--o{ PAYMENT_PROOFS : receives
  ORDERS ||--o{ INVENTORY_RESERVATIONS : owns
  SERVICE_REQUESTS ||--o{ QUOTATIONS : receives
  SERVICE_REQUESTS ||--o| PAYMENTS : may_create
  ORDERS ||--o{ RETURN_REQUESTS : has
  RETURN_REQUESTS ||--o| REFUNDS : creates
  COUPONS ||--o{ COUPON_REDEMPTIONS : records
```

Repositories validate every referenced ID and ownership before mutation because MongoDB does not enforce foreign keys. Historical records use snapshots so edits to users, products, categories, addresses, shipping rules, and payment settings do not rewrite history.

14. Indexing Strategy

14.1 Required indexes

Collection	Index
users	unique normalized email; unique canonical phone; status

Collection	Index
sessions	(userId, revokedAt, expiresAt); TTL on expiresAt
authTokens	unique token hash; TTL on expiresAt
addresses	(userId, isDefault)
roles	unique key
categories	unique slug; (isActive, isMvpEnabled, displayOrder)
products	unique slug; sparse unique ISBN if approved; (categoryId, isPublished, displayOrder); availability; searchable fields
carts	unique active userId; unique active sessionId; TTL on expiresAt
orders	unique reference; (customerId, submittedAt); (status, submittedAt); unique owner-scoped idempotency key
payments	(ownerType, ownerId); (status, updatedAt)
paymentProofs	unique (paymentId, submissionNumber); (status, createdAt)
inventoryReservations	partial unique active (orderId, orderItemId); (productId, variantId, status)
inventoryTransactions	(productId, variantId, createdAt); (sourceType, sourceId)
serviceCategories	unique slug; (isActive, displayOrder)
serviceRequests	unique reference; (customerId, createdAt); (status, createdAt)
quotations	(serviceRequestId, version) unique; (status, createdAt)
preOrders	unique reference; (productId, variantId, status); (customerId, createdAt)
returnRequests	unique reference; (orderId, createdAt)
refunds	(orderId, createdAt); (status, createdAt)
notifications	(recipientUserId, readAt, createdAt); unique dedupe key
coupons	unique normalized code; active/validity index
couponRedemptions	unique (couponId, orderId); (customerId, createdAt)
contentModules	(active, startsAt, endsAt, displayOrder)
shippingRules	(active, scopeType, scopeValue, priority)
settings	unique key
auditLogs	(entityType, entityId, createdAt); (actorId, createdAt)
outboxEvents	(status, availableAt); unique dedupe key

14.2 Search indexing

Use normalized exact fields and bounded query patterns first. A text index on searchText may be enabled only after Arabic behavior is validated. Do not promise ranking, autocomplete, or Atlas Search behavior in the API contract.

14.3 Index discipline

Index combinations must be justified by actual list/filter queries. Excessive indexes slow product and inventory writes. Use partial indexes for active/public records where safe.

15. Transactions & Concurrency

15.1 Transaction matrix

Operation	Transaction	Reason
Create order + payment + initial notifications/outbox	Yes	Avoid partial order/payment creation
Accept order + reserve every line	Yes	Keep order acceptance and stock aligned
Reject/cancel + release reservation	Yes	Keep state and stock aligned
Delivered/Picked Up + final deduction	Yes	Prevent duplicate deduction
Submit payment proof + payment/order state	Yes	Keep proof and state aligned
Admin payment review	Yes	State, audit, notification must agree
Accept quotation + activate payment	Yes	No payment before customer accepts
Approve return + initiate refund	Yes	Keep return/refund linkage aligned
Complete refund	Yes	Refund, return, audit, notification agree
Product content edit	No	Single-document update
Search/report/read	No	No cross-document write
Mark notification read	No	Single-document update

15.2 Inventory reservation transaction

For each order line, the service performs a conditional update:

```
const filter = {
  _id: productId,
  ...(variantId
    ? { "variants.variantId": variantId }
    : {}),
  $expr: {
    $gte: [
      { $subtract: ["$variants.stockTotal", "$variants.stockReserved"] },
      quantity,
    ],
  },
};

// Within a transaction:
// 1. Update stockReserved only if available >= quantity.
// 2. Insert inventoryReservations(active).
// 3. Insert inventoryTransactions(reservation).
// 4. Transition order from pending_review to accepted.
```

The actual Mongoose update must target the correct embedded variant with arrayFilters, or use a product-level atomic update for non-variant stock. If any line fails, the transaction aborts.

15.3 Concurrency rules

- Simultaneous Admin acceptance requires status: "pending_review" and the current order version.
- The second acceptance returns ORDER_STATE_CONFLICT.
- Inventory reservation never permits a negative stockTotal - stockReserved.
- Inventory adjustments require an expected version; stale Admin edits return INVENTORY_VERSION_CONFLICT.
- Checkout idempotency returns the original order result for the same owner-scoped key and request hash.
- A reused idempotency key with a different payload returns IDEMPOTENCY_KEY_REUSED.
- Pre-order allocation fairness remains unimplemented until 0D-11 is resolved.

Transactions do not solve every race automatically; predicates, unique indexes, version checks, and idempotency are required together.

16. Authentication Architecture

16.1 Access and refresh

- Access token: short-lived JWT used in Authorization: Bearer <token>.
- Refresh token: opaque random value stored in an HttpOnly, Secure, SameSite-configured cookie.
- MongoDB stores only a hash of the refresh token in sessions.
- Refresh rotates the token and invalidates the previous session token hash.
- Sessions can be individually revoked or globally revoked.
- JWT claims contain sub, role, sessionId, tokenVersion, iat, and exp; do not place sensitive profile data in claims.

16.2 Passwords and reset

- Hash with Argon2id.
- Password reset uses an opaque random token; store only its hash in authTokens.
- Reset tokens are one-time and TTL-limited.
- Do not use JWT as the password reset token.
- Do not log passwords, raw reset tokens, raw refresh tokens, or cookie values.

16.3 Example JWT service

```
type AccessClaims = {
  sub: string;
  role: "customer" | "admin" | "owner";
  sessionId: string;
  tokenVersion: number;
};

export function issueAccessToken(claims: AccessClaims): string {
  return jwt.sign(claims, env.JWT_ACCESS_SECRET, {
    algorithm: "HS256",
    expiresIn: env.JWT_ACCESS_TTL,
    issuer: env.JWT_ISSUER,
    audience: env.JWT_AUDIENCE,
  });
}
```

The production signing algorithm and secret storage are environment-controlled. Secrets never belong in settings or source control.

17. Authorization & RBAC

17.1 Roles

Role	Meaning
Guest	Unauthenticated visitor; may browse, maintain a guest cart, checkout, and view an order with a guest token
Customer	Registered customer; may access only their own account, orders, payments summaries, services, pre-orders, returns, and notifications
Admin	Operational user with configured permissions for Admin modules
Owner	Highest initial operational role; equal or greater privilege than Admin over every module

17.2 Permission model

Permission keys follow `<module>.<action>`, for example `orders.read`, `orders.accept`, `payments.review`, `products.write`, `settings.write`, and `reports.read`.

17.3 Permission matrix

Operation	Guest	Customer	Admin	Owner
Browse published catalog/categories	Yes	Yes	Yes	Yes
Search/filter catalog	Yes	Yes	Yes	Yes
Add/update guest cart	Yes	Yes	Optional	Optional
Submit product order	Yes	Yes	No by default	No by default
View own order/service by ownership	Guest token	Yes	No ownership restriction	No ownership restriction
Register/login/reset	Yes	Yes	Yes	Yes
Read Admin dashboard	No	No	Permission	Yes
Read/update products/categories	No	No	Permission	Yes
Accept/reject orders	No	No	<code>orders.write</code>	Yes
Review payment proofs	No	No	<code>payments.review</code>	Yes
Adjust inventory/shipping	No	No	Permission	Yes
Manage services/quotations	No	No	Permission	Yes
Approve returns/record refunds	No	No	Permission	Yes
Read reports	No	No	<code>reports.read</code>	Yes
Manage roles/settings	No	No	Explicit permission if granted	Yes
Create Admin	No	No	No	Owner only

The matrix describes the minimum boundary. Admin permissions are stored in roles; the initial UI need not expose future specialized roles.

17.4 Middleware

```
export const requirePermission =
  (permission: string) =>
    (req: AuthenticatedRequest, _res: Response, next: NextFunction) => {
      if (!req.auth?.permissions.includes(permission)) {
        return next(new AppError("FORBIDDEN", "Permission denied", 403));
      }
      next();
    };
```

Record ownership is checked in the service, not inferred from a route parameter.

18. User & Account Management

18.1 Registration workflow

1. Validate name, phone, unique valid email, and password.
2. Normalize email/phone and check uniqueness.
3. Hash password with Argon2id.
4. Create user with `emailVerifiedAt: null, role: customer, status: active`.
5. Generate hashed email-verification token.
6. Commit user, token, notification/outbox event in a transaction where multiple records are created.
7. Send verification email through the worker.
8. Allow login before verification if the source policy permits; gate only actions defined by `auth.verificationPolicy`.

18.2 Login workflow

1. Find by normalized email or canonical phone.
2. Verify active status and Argon2id hash.
3. Apply account/email throttling before expensive hash checks when appropriate.
4. Create session with hashed refresh token.
5. Issue access token and set refresh cookie.
6. Update `lastLoginAt` without blocking login if the metadata update fails.

18.3 Verification and password reset

- Verification consumes one valid token and sets `emailVerifiedAt`.
- Reset request returns a neutral response whether the email exists.
- Reset consumes the token, replaces the hash, and revokes sessions.
- Expired/consumed tokens return `AUTH_TOKEN_INVALID` without revealing account existence.

18.4 Account merge

A guest order may be associated with a registered account only after the account's email is verified and the guest access proof matches. Merge is transactional and audited; it does not overwrite order snapshots.

19. Guest Checkout

19.1 Guest access

On order creation, generate a high-entropy guest order token. Store only its hash on the order. Return the raw token once to the client and use it in a secure order-access link or explicit header. Do not expose the token in logs, analytics, WhatsApp deep-link text, or notification bodies.

19.2 Guest workflow

```
flowchart LR
  Browse --> Cart --> Customer[Contact + address]
  Customer --> Fulfillment[Delivery or pickup]
  Fulfillment --> Payment[Payment selection]
  Payment --> Proof{Proof required?}
  Proof -->|Yes| Upload[Cloudinary proof + metadata]
  Proof -->|No| Review[Review totals]
  Upload --> Review
  Review --> Submit[POST /orders]
  Submit --> Pending[Pending Review + guest token]
```

Guest checkout requires recipient name and phone. Email is optional but enables email confirmation when supplied. It never forces account registration.

19.3 Guest security

- Rate-limit guest order access and order creation by IP/device signals.
- Require both order reference and guest token for access.
- Return the same not-found/unauthorized shape for invalid combinations.
- Never allow guest clients to list orders or access payment proof review data.

20. Cart Architecture

20.1 Lifecycle

- Guest carts are keyed by a secure session identifier and expire with TTL.
- Registered carts are keyed by user and persist across sessions/devices.
- Login merges the guest cart into the user cart using deterministic conflict handling.
- The cart contains fixed-price products/variants only.
- Cart display prices are snapshots, not authoritative financial values.

20.2 Add/update rules

1. Validate product is published and category is MVP-enabled.
2. Validate variant selection when variants exist.
3. Reject services and inactive products.
4. Normalize quantity to an integer ≥ 1 .
5. Recalculate a display snapshot from current catalog data.
6. Increment cart version on mutation.

20.3 Checkout revalidation

At checkout, read current product/variant, availability, price, and category. Recalculate totals server-side and return a structured change set if price or availability differs. The client must show the change before the order can be submitted.

20.4 Cart merge

Merge items by `productId + variantId`. Quantities are added only if current catalog state permits; otherwise return a conflict list for customer review. Never silently drop an item.

21. Product & Category APIs

21.1 Product rules

- Product has name, category, price, availability, and image(s) at minimum.
- Description may be null and does not block publication.
- Variants are optional and independently carry price/availability.
- Books are structurally and visually prioritized by `isBooksCore` and `displayOrder`.
- Reviews and ratings do not exist.
- Future categories remain inactive/non-purchasable in MVP.

21.2 Public read model

Public product responses include only published, MVP-enabled data: `slug`, localized name/description, images, applicable metadata, variant choices, price, availability, and pre-order eligibility. Do not expose stock counters, internal IDs when unnecessary, audit records, or payment data.

21.3 Admin write model

Admin product writes validate category ownership, image metadata, price, variant uniqueness, stock non-negativity, and publication rules. A description is optional. Every price and availability mutation creates an audit record.

21.4 Category rules

`GET /categories` returns active/MVP-enabled categories ordered books-first. `POST/PATCH/DELETE` are Admin operations; normal deletion is deactivation and blocked when historical products depend on the category.

22. Inventory Architecture

22.1 Stock model

For each product or variant:

```
available = stockTotal - stockReserved
```

stockReserved is incremented only when Admin accepts an order. stockTotal is deducted only when the order reaches Delivered or Picked Up. Pre-orders do not reserve stock.

22.2 Inventory state effects

Trigger	Effect
Order submitted	No stock change
Admin accepts	Reserve requested quantity
Admin rejects	No reservation; release any existing reservation defensively
Customer cancels pending review	No reservation in normal path
Fulfillment reaches delivered/picked up	Deduct total and reserved; consume reservation
Pre-order accepted	No stock movement
Return/restock	Apply only if approved policy and physical restock is recorded

22.3 Admin adjustment

Inventory adjustments require the current document version, a non-empty reason, actor identity, and an append-only ledger row. Never edit an old ledger row to correct a mistake; append a correction.

22.4 Open inventory behavior

Concurrent last-unit behavior (0D-08) uses safe first-successful conditional reservation in the initial technical implementation. It does not promise fairness or queue order. Partial acceptance (0D-21) is not silently implemented; the order is accepted only if all lines can be reserved unless the owner approves another rule.

23. Checkout Architecture

23.1 Stages

1. Customer information.
2. Delivery address or pickup.
3. Payment method.
4. Payment proof where required.

5. Coupon.
6. Server-calculated review.
7. Submit with idempotency protection.

23.2 Address

Delivery requires governorate, city, area, street, building number, floor, apartment, landmark, recipient name, recipient phone, and additional notes as applicable. Pickup has no address requirement but displays configured library location.

23.3 Fulfillment and shipping

- Pickup is always 0 EGP and uses no shipping rule.
- Delivery requires a matching serviceable `shippingRules` record and shows its estimate.
- Customer never selects the carrier.
- Admin later chooses the carrier and may record a final adjusted cost.
- Delivery promises and excluded-area lists remain hidden until approved.

23.4 Payment selection

The API reads enabled methods and payment details from `settings.payment.methods` and `settings.payment.details`. It does not assume both source lists are approved. A non-COD method creates a payment with `proofRequired: true`; COD does not.

23.5 Create-order transaction

Inside one transaction:

1. Acquire owner-scoped idempotency key.
2. Re-read cart lines and current catalog values.
3. Validate fulfillment and payment configuration.
4. Calculate coupon and shipping estimate.
5. Snapshot customer/address/product/payment facts.
6. Create order in `pending_review`.
7. Create payment if required by configuration.
8. Create notification/outbox events.
9. Clear or version the cart.
10. Commit and return order reference plus guest token where applicable.

No stock reservation occurs at order creation.

24. Order Management

24.1 Order state machine

```
stateDiagram-v2
    [*] --> pending_review
    pending_review --> accepted: Admin accepts
    pending_review --> rejected: Admin rejects
    pending_review --> cancelled: Customer cancels
    accepted --> awaiting_payment: digital payment
    awaiting_payment --> payment_verification: proof submitted
    payment_verification --> payment_confirmed: Admin confirms
    payment_verification --> awaiting_new_proof: Admin requests/rejects
    awaiting_new_proof --> payment_verification: customer re-uploads
    accepted --> customer_confirmation_required: COD
    customer_confirmation_required --> confirmed: customer confirms
    payment_confirmed --> preparing
    confirmed --> preparing
    preparing --> ready_for_pickup: pickup
    ready_for_pickup --> picked_up
    picked_up --> completed
    preparing --> shipped: delivery
    shipped --> out_for_delivery
    out_for_delivery --> delivered
    delivered --> completed
    completed --> returned: approved return
```

24.2 Transition ownership

Transition	Actor
Create pending_review	System
Accept/reject	Admin/Owner with permission
Cancel from pending review	Customer
Upload proof	Order owner
Confirm/reject/request new proof	Admin/Owner with payment permission
COD confirmation	Customer
Preparing and fulfillment states	Admin/Owner with order/shipping permission
Return transition	Return workflow, Admin/Owner

24.3 Editing and cancellation

Customer editing is restricted to `pending_review`, revalidates current price/availability, and increments order version. Cancellation is allowed only from `pending_review`. No post-payment change may alter financial totals. Unspecified post-acceptance non-financial edits remain blocked by default and recorded as `OPEN / NOT SPECIFIED`.

24.4 Order snapshots

Every order stores line product/variant name, image, category, selected attributes, quantity, unit price, line total, availability at submission, stock key, contact snapshot, address snapshot, payment method snapshot, coupon snapshot, and shipping estimate/final values.

25. Payment Architecture

25.1 Payment and order are separate

The order has an order lifecycle. The payment has a payment lifecycle. Fulfillment has its own shipping/pickup status. A payment confirmation may permit an order transition but does not replace the order status.

25.2 Payment states

```
stateDiagram-v2
    [*] --> not_submitted
    not_submitted --> proof_uploaded: proof stored
    proof_uploaded --> under_review: submission accepted
    under_review --> confirmed: Admin confirms
    under_review --> rejected: Admin rejects
    under_review --> new_proof_requested: Admin requests new proof
    new_proof_requested --> proof_uploaded: customer resubmits
    rejected --> proof_uploaded: configured retry
```

rejected and new_proof_requested are distinct: rejection may be terminal for a submission, while the order can still request another proof.

25.3 Payment methods

Payment method definitions are configurable and snapshotted. The source discrepancy is 00-20; production seed data must not enable an unapproved method. There is no transaction-ID customer input and no automated gateway.

25.4 COD

COD creates no proof requirement. After Admin accepts the order, the customer must explicitly confirm before confirmed. No automatic timeout is invented.

26. Payment Proof Workflow

26.1 Submission

1. Confirm the authenticated customer or guest order-token ownership.
2. Confirm order/payment is in a proof-eligible state.
3. Request a signed Cloudinary upload configuration or upload result according to the approved direct-upload flow.
4. Validate returned metadata, file type, size, resource type, ownership context, and Cloudinary folder.
5. Create a new paymentProofs document with a sequential submissionNumber.
6. In one transaction update payment to under_review, order to payment_verification, create audit and outbox/notification records.
7. Return proof status, not raw private delivery URLs.

26.2 Admin review

Admin can Confirm, Reject, Request New Proof, or add an internal note where the configured action supports it. The transaction updates payment/proof/order state, writes audit, and creates customer/Admin notifications.

26.3 Security

- Proof documents are private by default.
- Admin/Owner receives a short-lived signed URL on demand.
- Customer sees only their own proof status and allowed review note.
- Raw Cloudinary signatures, tokens, and permanent URLs never enter logs or customer responses.
- Cloudinary deletion follows approved retention and audit policy; never delete a proof merely because a new proof is uploaded.

27. Shipping Architecture

27.1 shippingRules

The approved backend plan uses a dedicated `shippingRules` collection for configurable estimated shipping. Rules match from specific to general:

1. area,
2. city,
3. governorate,
4. default.

Within the same scope, the highest priority active rule wins. Effective date and serviceability are checked. If no rule matches, return `SHIPPING_CONFIGURATION_UNAVAILABLE`; do not invent a price.

27.2 Estimate

`POST /checkout/shipping-estimate` accepts fulfillment method and address fields. Pickup returns zero and the configured pickup location. Delivery returns estimate, currency, matched-rule reference, and `serviceable`. The client does not receive internal rule details beyond safe source labels.

27.3 Admin carrier/final cost

Admin selects a carrier from configured/provider text and may record a final shipping cost. The order stores:

- customer-facing estimate,
- estimate rule snapshot,
- provider chosen by Admin,
- final cost,
- adjustment reason,
- timestamp and actor,
- whether customer reconfirmation is required once `OD-04` is resolved.

Until the binding point is approved, a changed final amount is not represented as customer acceptance.

27.4 Delivery state

Delivery uses preparing → shipped → out_for_delivery → delivered. Pickup uses preparing → ready_for_pickup → picked_up. No delivery-time promise is returned unless configured and approved.

28. Coupon Architecture

28.1 Calculation

Coupon validation is centralized and server-side. It accepts the current order snapshot and returns:

```
type CouponResult = {
  valid: boolean;
  code: string;
  discountMinor: number;
  reasonCode?: string;
  appliedTo: "order" | "product" | "category";
};
```

Supported confirmed capabilities are percentage discounts, fixed discounts, seasonal offers, product/category scope, and Admin management.

28.2 Open governance

Stacking, usage limits, expiry defaults, minimum order, and customer restrictions remain configurable under 00-17. A coupon code does not imply any rule. Null policy values mean the API must not apply that policy.

28.3 Consumption

1. Validate without consuming during review.
2. Recalculate inside order creation.
3. Create couponRedemptions and increment usage only inside the create-order transaction.
4. Use unique indexes and owner-scoped idempotency to prevent double consumption.
5. Define cancellation/refund coupon treatment in settings before applying production reversals.

29. Student Services Architecture

29.1 Domain boundary

Student services are not fixed-price catalog products and never enter carts or orders. They use serviceCategories, serviceRequests, quotations, and optionally a payment linked to an accepted quotation.

Confirmed service categories:

- Printing.
- Photocopying.
- Binding.

- Applications and transfers.
- Research/formatting help.
- Other administrative services only when approved.

29.2 Service request flow

```

flowchart LR
    Category[Service category] --> Request[Submit request]
    Request --> Review[Admin review]
    Review --> Quote[Quotation]
    Quote --> Decision{Customer decision}
    Decision -->|Accept| Payment[Payment]
    Decision -->|Reject| Closed[Closed - not proceeding]
    Payment --> Processing[Processing]
    Processing --> Complete[Completed]
    Review --> Declined[Closed - declined]

```

29.3 Form configuration

The API renders only configured fields from the versioned service category. Until 00-12 is approved, fields remain configurable and the API accepts a bounded description plus no invented document/file field. Applications/transfers copy and data must represent that the library performs the administrative service on the customer's behalf.

Files are exchanged via WhatsApp/Telegram. The request schema, route parser, and Cloudinary integration must reject attachments, multipart service uploads, and arbitrary file metadata.

29.4 Service payment

Payment is unavailable before quotation acceptance. Reusing the product proof workflow after acceptance is the approved technical direction, but COD applicability is 00-14.

30. Quotations

30.1 Quote lifecycle

```

stateDiagram-v2
    [*] --> draft
    draft --> sent: Admin sends
    sent --> accepted: customer accepts
    sent --> rejected: customer rejects
    accepted --> payment

```

Expiry is not implemented until a policy is approved.

30.2 Accept transaction

Within one transaction:

1. Update quotation from sent to accepted with version predicate.
2. Set customerDecisionAt.
3. Update service request to awaiting_payment.

4. Create or activate payment using the quote amount snapshot.
5. Write audit and notification/outbox.

A repeated acceptance returns the original result; a different version returns `QUOTE_STATE_CONFLICT`.

30.3 Reject transaction

Update quote to rejected, service request to `closed_not_proceeding`, write the decision note if provided, and notify Admin. No payment is created.

31. Preorders

31.1 Lifecycle

```
stateDiagram-v2
    [*] --> requested
    requested --> admin_review
    admin_review --> accepted
    admin_review --> rejected
    accepted --> payment_pending
    payment_pending --> payment_verification
    payment_verification --> confirmed
    confirmed --> available
    available --> fulfilled
    requested --> cancelled
```

31.2 Rules

- Only out-of-stock, pre-order-eligible products/variants can create a pre-order.
- Capture product/variant snapshot, quantity, price at request, and expected availability only if Admin entered it.
- Acceptance creates no inventory reservation.
- Once available and paid/confirmed, the record hands off to standard fulfillment from preparing.
- Price-honor behavior if catalog price changes remains 0D-10.
- Allocation order across multiple pre-orders remains 0D-11; do not assign a fairness sequence.

32. Returns

32.1 Eligibility

Return requests start from completed orders. The customer selects eligible item(s), quantity, reason, and evidence where the configured policy permits. Wrong-item/damaged-item is the confirmed policy case; category-specific periods and exclusions remain 0D-15.

32.2 Lifecycle

```
stateDiagram-v2
    [*] --> return_requested
    return_requested --> return_review
    return_review --> return_approved
    return_review --> return_rejected
    return_approved --> refund_initiated
    refund_initiated --> refund_completed
```

32.3 Rules

- Customer owns only their order and return records.
- Admin/Owner reviews and approves/rejects.
- No refund is implied by approval until a separate refund record is initiated.
- Evidence metadata must obey configured retention and privacy rules.
- Inventory restock is a separate controlled action and must not be assumed merely because a return was approved.

33. Refunds

Refunds are manually executed by Admin through InstaPay or an approved e-wallet. The backend records the result; it does not call a gateway.

33.1 Completion transaction

1. Confirm return is `refund_initiated`.
2. Confirm the refund has not already been completed.
3. Validate amount does not exceed the refundable amount.
4. Create/update the distinct refunds record with method, amount, timestamp, and internal reconciliation reference.
5. Update return to `refund_completed`.
6. Write audit and notify the customer.

The transaction is idempotent. A second completion returns the existing refund or `REFUND_ALREADY_COMPLETED`.

33.2 Partial refunds

Item-level return requests support a bounded item quantity. Partial financial behavior beyond that requires owner-approved policy; do not invent prorating, coupon reversal, or shipping refund logic.

34. Notifications

34.1 Notification channels

Channel	Use
In-app	Persisted order, payment, service, pre-order, return, and operational lifecycle events
Email	Verification, password reset, and relevant transactional messages
Socket.IO	Real-time delivery convenience
WhatsApp	Manually initiated contextual support only; not automated notification

34.2 Event examples

- `user_registered`
- `email_verified`
- `order_created`
- `order_accepted`
- `order_rejected`
- `payment_proof_submitted`
- `payment_verified`
- `payment_new_proof_requested`
- `order_confirmed`
- `order_fulfillment_changed`
- `service_requested`
- `quotation_sent`
- `quotation_decision_recorded`
- `preorder_status_changed`
- `return_status_changed`
- `refund_completed`

34.3 Notification creation

State-changing application services create notification records and outbox events in the same transaction. Use deterministic dedupe keys such as `order:<id>:status:<status>:<version>`. A failed email or socket delivery updates delivery status and is retried; it never changes the authoritative business state.

35. Socket.IO Architecture

35.1 Authentication

The client sends an access token during the Socket.IO handshake. The server verifies signature, expiration, session/version status, and role. A disconnected or revoked session is rejected.

35.2 Rooms

- `user:{userId}` for the authenticated customer.
- `order:{orderId}` only for the order owner or authorized Admin/Owner.
- `service:{serviceId}` only for the request owner or authorized Admin/Owner.
- Admin/Owner operational room is permission-gated; do not use a universal unrestricted admin room.

35.3 Events

Use namespaced event names:

```
notification.created
order.status_changed
payment.proof_submitted
service.request_created
service.quotation_sent
admin.queue_changed
```

Events contain a safe entity reference, event ID, version, and minimal display payload. The client refreshes authoritative data after reconnect or version mismatch.

35.4 Failure behavior

If Socket.IO is unavailable, the mutation still commits. On reconnect, the client reads persisted notifications and record state. Unauthorized room joins return a generic forbidden error without revealing record existence.

36. Email Architecture

36.1 Adapter

```
export interface EmailService {
  send(input: {
    to: string;
    template:
      | "verify_email"
      | "password_reset"
      | "order_confirmation"
      | "payment_update"
      | "service_update";
    locale: "ar" | "en";
    variables: Record<string, string | number>;
    dedupeKey: string;
  }): Promise<{ providerMessageId?: string }>;
}
```

The implementation uses SMTP, Gmail, or a private mail server configured through environment variables. Do not make Resend/Brevo a required provider.

36.2 Templates

- Verification link.
- Password reset link.
- Order submitted/accepted/rejected/fulfillment update.

- Payment proof requested/confirmed/rejected.
- Service request/quotation/processing/completion.
- Admin operational notifications where configured.

36.3 Delivery

Email work is created in `outboxEvents`, processed by the worker, retried with backoff, and marked terminal after the configured maximum. Templates must not include payment proofs, raw secrets, or unsupported delivery promises.

37. Cloudinary Architecture

37.1 Approved flow

1. Client requests a signed upload configuration for an allowed context.
2. Backend signs a constrained folder/preset, resource type, timestamp, and public ID policy.
3. Client uploads directly to Cloudinary.
4. Client submits the resulting metadata to the backend.
5. Backend validates the signature/result shape, ownership context, file type, byte size, and folder.
6. Backend stores only metadata in `paymentProofs` or `product media` fields.

37.2 Media policy

- Product media may be public or transformed for delivery according to the configured product-media policy.
- Payment proofs are private and served through short-lived signed URLs.
- Service attachments are prohibited.
- Allowed payment proof types: PNG, JPEG, and WebP unless operations configures a narrower set.
- Enforce a bounded file-size limit in both signed policy and backend metadata validation; the exact production limit is configurable.
- Store `publicId`, resource type, format, bytes, width/height, and optional hash.

37.3 Replacement and orphan handling

Replacement creates a new metadata record or new product image entry before the old asset is removed. A cleanup job can identify Cloudinary assets not referenced by active MongoDB metadata, but deletion requires a retention-safe review. Never delete a payment proof merely because it is no longer the newest submission.

38. WhatsApp Integration

38.1 MVP

Generate normal WhatsApp links with URL-encoded, safe prefills. Do not require the WhatsApp Business API.

```
export function buildWhatsAppUrl(
  phone: string,
  context?: { product?: string; orderReference?: string; serviceReference?: string },
): string {
  const parts = ["السلام عليكم، أحتاج مساعدة من مكتبة الأزهر"];
  if (context?.product) parts.push(`المنتج: ${context.product}`);
  if (context?.orderReference) parts.push(`رقم الطلب: ${context.orderReference}`);
  if (context?.serviceReference) parts.push(`رقم الخدمة: ${context.serviceReference}`);
  return `https://wa.me/${encodeURIComponent(phone)}?text=${encodeURIComponent(parts.join("\n"))}`;
}
```

Only safe public references are included. Never include payment proofs, addresses, tokens, or private notes.

38.2 Contextual actions

Product, cart/checkout, order detail, service request, Admin customer/order/service detail, and contact page may expose a WhatsApp action. The copy must make clear that the chat does not create, change, confirm, or cancel a record.

38.3 Future adapter

Keep a `WhatsAppLinkAdapter` interface so a future Business API can be added without changing order/service domain logic.

39. Outbox Architecture

39.1 Event contract

```
type OutboxEvent = {
  eventType: string;
  aggregateType: "order" | "payment" | "service" | "user" | "return";
  aggregateId: ObjectId;
  payload: Record<string, unknown>;
  dedupeKey: string;
  status: "pending" | "processing" | "sent" | "failed";
  attempts: number;
  availableAt: Date;
  processedAt?: Date;
  lastError?: string;
};
```

39.2 Lifecycle

1. Business transaction inserts pending event with a unique dedupe key.
2. Worker claims events with a lease/processing timestamp.
3. Adapter performs the delivery idempotently.
4. Worker marks sent with `processedAt`, or reschedules with backoff.
5. Terminal failures remain failed with a safe error summary for Admin inspection.

39.3 Crash safety

Processing claims must expire. A worker crash cannot leave an event permanently stuck in processing; the next poll reclaims expired work. Delivery adapters must tolerate replay.

40. Background Jobs

The same Node.js process runs an internal worker. It must not become a separate required service for MVP.

40.1 Worker responsibilities

- Claim pending outbox events.
- Deliver email.
- Emit Socket.IO notifications.
- Retry failed events.
- Reclaim expired processing leases.
- Run cleanup tasks through the scheduler.

40.2 Backoff

Use bounded exponential backoff with jitter, for example 30s, 2m, 10m, 30m, then terminal/configured review. Store `attempts` and `availableAt`; do not retry indefinitely.

40.3 Idempotency

Every event has a deterministic dedupe key. The worker checks sent before repeating work and adapters accept the dedupe key where provider APIs support it.

41. Cron Jobs

Node cron runs only idempotent, bounded tasks:

Job	Action
Outbox poll	Claim and deliver due events
Lease recovery	Requeue stale processing events
Guest cart cleanup	TTL is primary; job can report anomalies
Auth token cleanup	TTL is primary; job can report anomalies
Notification archival	Archive only after approved retention
Expired content	Hide date-windowed modules
Reservation review	Report or release only when an approved expiry policy exists
Orphan media scan	Report unreferenced Cloudinary metadata; deletion needs policy
Report health	Record job duration/failure metrics

No reservation expiry duration is hard-coded until the owner resolves the configured policy. Cron jobs must use distributed/lease-safe claims so two app instances do not repeat a mutation.

42. Search & Filtering

42.1 Searchable fields

Product name, author, grade, stage, subject, publisher, ISBN, and category. Filters: education type, stage, grade, subject, publisher, price range, and availability.

42.2 Normalization

- Trim and Unicode-normalize text.
- Store a lowercased search representation.
- Preserve Arabic text; do not transliterate as a requirement.
- Normalize ISBN by removing spaces/hyphens where appropriate.
- Canonicalize slugs.
- Treat null metadata as absent, not as a reason to hide a product.

42.3 Query plan

1. Apply `isPublished: true` and active/MVP category constraints.
2. Apply exact filters.
3. Apply bounded field-specific search or validated text query.
4. Apply approved deterministic sort, defaulting to books-first display order where configured.
5. Paginate with `limit` and a stable cursor or page contract.
6. Return active filters and a result count when the query can provide it efficiently.

Do not add autocomplete, advanced ranking, or an external search engine in MVP.

43. Reports & Analytics

43.1 Reports

Admin/Owner reports cover:

- Orders and order status.
- Revenue.
- Orders originating outside Qena.
- Payment-method distribution.
- Service requests and quotation conversion.
- Product/category demand.
- Pre-order demand.
- Coupon usage.

Reports use aggregation pipelines with date/status/geography filters. KPI targets are not invented.

43.2 Business events

Emit events only where they represent a real confirmed workflow:

user_registered, email_verified, order_created, payment_submitted, payment_verified, order_confirmed, order_fulfilled, order_cancelled, service_requested, service_completed, coupon_used, return_requested, refund_completed.

Events may be stored through the outbox or a bounded analytics/event collection only if the implementation needs it. Do not add an external analytics platform as an MVP dependency.

44. Audit Logging

44.1 Audited actions

- Order accept/reject/status changes.
- Payment proof confirm/reject/request-new-proof.
- Inventory adjustments and stock deduction.
- Shipping provider/final-cost changes.
- Quotation creation/send/accept/reject.
- Return decisions.
- Refund initiation/completion.
- Role/permission changes.
- Settings changes.
- Product price/availability/publication changes.
- Coupon activation and rule changes.

44.2 Audit record

Record actor, role, action, target type/ID, previous state, new state, reason where applicable, request ID, hashed IP where approved, and timestamp. Exclude passwords, tokens, raw proof URLs, and unnecessary PII.

Audit records are append-only. No user-facing edit/delete API exists.

45. Validation

45.1 Zod boundary

Every request body, path parameter, query string, and selected header is parsed with Zod before the controller calls a service.

```
const CreateOrderSchema = z.object({
  customer: z.object({
    name: z.string().trim().min(1).max(160),
    phone: z.string().trim().min(7).max(32),
    email: z.string().email().optional(),
  }),
  fulfillment: z.discriminatedUnion("method", [
    z.object({ method: z.literal("pickup") }),
    z.object({
      method: z.literal("delivery"),
      address: DeliveryAddressSchema,
    }),
  ]),
  paymentMethodKey: z.string().trim().min(1),
  couponCode: z.string().trim().max(64).optional(),
});
```

45.2 Validation rules

- Reject negative or fractional money/quantity/stock values.
- Require amountMinor and currency together.
- Require variant when the product has variants.
- Reject service IDs in cart/order product lines.
- Require proof for configured non-COD methods and reject proof for COD.
- Reject customer writes to authoritative status, payment confirmation, stock, role, or audit fields.
- Enforce ownership on customer-scoped queries.
- Use allowlists for sort/filter fields.
- Sanitize or reject MongoDB operator injection in string inputs.

46. Error Handling

46.1 Error hierarchy

```
export class AppError extends Error {
  constructor(
    public readonly code: string,
    message: string,
    public readonly status: number,
    public readonly details?: unknown,
    public readonly retryable = false,
  ) {
    super(message);
  }
}

export class ConflictError extends AppError {
  constructor(code: string, message: string, details?: unknown) {
    super(code, message, 409, details);
  }
}
```

46.2 Error codes

Code	HTTP	Meaning
VALIDATION_ERROR	400	Request shape/value invalid
AUTH_REQUIRED	401	Missing/invalid access token
AUTH_INVALID_CREDENTIALS	401	Login failed
FORBIDDEN	403	Permission or ownership denied
NOT_FOUND	404	Safe resource-not-found response
RESOURCE_CONFLICT	409	Version/state conflict
ORDER_STATE_CONFLICT	409	Invalid order transition
INSUFFICIENT_STOCK	409	Reservation cannot be made
IDEMPOTENCY_KEY_REUSED	409	Same key used with another payload
PAYMENT_PROOF_REQUIRED	422	Required proof missing
SHIPPING_CONFIGURATION_UNAVAILABLE	422	No approved shipping rule
BUSINESS_RULE_VIOLATION	422	Confirmed rule prevents action
RATE_LIMITED	429	Throttle applied
INTERNAL_ERROR	500	Unexpected failure
DEPENDENCY_UNAVAILABLE	503	External adapter unavailable

Do not leak Mongoose errors, stack traces, provider credentials, or whether a sensitive account exists.

47. API Response Standards

47.1 Success

```
{
  "success": true,
  "data": {
    "reference": "ORD-20260923-0001",
    "status": "pending_review"
  },
  "meta": {
    "requestId": "req_01J...",
    "pagination": null
  }
}
```

47.2 Error

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Some fields need attention.",
    "fields": [
      { "path": "fulfillment.address.governorate", "code": "required" }
    ]
  },
  "meta": {
    "requestId": "req_01J..."
  }
}
```

47.3 Pagination

```
{
  "success": true,
  "data": [],
  "meta": {
    "requestId": "req_01J...",
    "pagination": {
      "limit": 24,
      "nextCursor": "eyJjcmVhdGVkQXQiOiIyMDI2..."
    }
  }
}
```

Stable cursor pagination is preferred for changing operational queues. Page-number pagination may be used for public catalog if the API contract documents its consistency behavior.

47.4 HTTP conventions

200 read/update, 201 create, 202 accepted for asynchronous delivery where relevant, 204 idempotent delete/deactivate without body, 400 validation, 401 authentication, 403 authorization, 404 safe not found, 409 conflict, 422 business rule, 429 rate limit, 500 unexpected, 503 dependency unavailable.

48. API Endpoint Specification

All routes are under `/api/v1`. Unless stated otherwise, responses use the envelope in Section 47 and include a request ID.

48.1 Authentication and account endpoints

Method / route	Auth / permission	Request	Success	Errors / side effects
POST /auth/register	Guest	name, phone, email, password	201 safe user plus verification state	400/409; creates user, hashed verification token, email outbox
POST /auth/login	Guest	email or phone, password	200 access token, refresh cookie, session summary	401/429; creates session, updates last login

Method / route	Auth / permission	Request	Success	Errors / side effects
POST /auth/refresh	Refresh cookie	no body	200 rotated access token and cookie	401; rotates session token hash
POST /auth/logout	Authenticated	optional current/global scope	204	401; revokes session; audit for global revoke
POST /auth/verify-email	Token	token	200 verified user state	400/410; consumes token, notification
POST /auth/forgot-password	Guest	email	202 neutral response	400/429; reset outbox if account exists
POST /auth/reset-password	Token	token, new password	204	400/410; updates hash, revokes sessions
GET /me	Customer/Admin/Owner	none	200 safe profile and role	401; no sensitive hashes

48.2 Public catalog endpoints

Method / route	Auth / permission	Request	Success	Errors / side effects
GET /categories	Public	active filters, locale	200 books-first categories	500; read-only
GET /products	Public	query, category, filters, cursor/limit	200 public product cards	400; read-only
GET /products/:slug	Public	slug	200 product detail	404; read-only
GET /search	Public	q and documented filters	200 result page	400; bounded query
GET /content/home	Public	locale, current date	200 active modules	500; date-window filtering

48.3 Cart and checkout endpoints

Method / route	Auth / permission	Request	Success	Transaction / side effects
GET /cart	Guest session or Customer	cart context	200 cart and version	Read-only
POST /cart/items	Guest session or Customer	productId, variantId, quantity	201/200 updated cart	Validates product/variant; no stock reservation
PATCH /cart/items/:itemId	Owner	quantity, expectedVersion	200 updated cart	Version conflict on stale write
DELETE /cart/items/:itemId	Owner	expectedVersion	204	Idempotent removal
POST /checkout/shipping-estimate	Guest/Customer	fulfillment + address	200 estimate/serviceability	Read-only rule lookup

Method / route	Auth / permission	Request	Success	Transaction / side effects
POST /orders	Guest/Customer	contact, fulfillment, payment method, coupon, idempotency key	201 order reference/status/access token	Transaction: revalidate, snapshot, create order/payment, redemption, notification/outbox; audit create where configured
GET /orders/:reference	Guest token or Customer ownership or Admin permission	reference and guest token if guest	200 order-safe detail	404/403; never proof internals
PATCH /orders/:reference	Customer ownership	editable pending-review fields, expectedVersion	200 updated order	Transaction: revalidate totals; only pending review
POST /orders/:reference/cancel	Customer ownership	expectedVersion	200 cancelled order	Transaction: pending-review only; notification/audit

POST /orders contract

Authentication: guest session or customer access.

Permission: customer checkout; Admin does not use this route to create operational orders.

Validation: at least one cart line; current product/variant eligible; fulfillment valid; payment method enabled; delivery serviceable; no service line; coupon valid; proof requirements satisfied where proof is submitted.

Success: order begins pending_review, with reference, safe status, totals, payment next step, and a guest access token only for guest order creation.

Failure: PRICE_CHANGED, AVAILABILITY_CHANGED, SHIPPING_CONFIGURATION_UNAVAILABLE, PAYMENT_METHOD_UNAVAILABLE, PAYMENT_PROOF_REQUIRED, or validation error.

Side effects: cart cleared/versioned; in-app/Admin notification and email outbox where email exists.

Audit: record order creation metadata without secrets.

48.4 Payment endpoints

Method / route	Auth / permission	Request	Success	Transaction / side effects
GET /orders/:reference/payment	Owner/Admin	reference	200 payment instructions/state	Proof metadata only
POST /orders/:reference/payment-proof/upload-config	Order owner/Admin where delegated	context, file metadata	200 constrained signed upload config	No business state mutation
POST /orders/:reference/payment-proofs	Order owner	Cloudinary metadata, note, idempotency key	201 proof status	Transaction: create proof, payment under review, order state, audit/outbox
POST /admin/payments/:paymentId/confirm	payments.review	expectedVersion	200 updated state	Transaction: confirm proof/payment/order, audit/outbox
POST /admin/payments/:paymentId/reject	payments.review	reason, expectedVersion	200 updated state	Transaction: reject/request new proof according to action, audit/outbox

Method / route	Auth / permission	Request	Success	Transaction / side effects
POST /admin/payments/:paymentId/request-new-proof	payments.review	note, expectedVersion	200 updated state	Transaction: new-proof state, audit/outbox

Proof review responses never expose raw signed URLs to unauthorized users. Confirm/reject requests require current payment/proof state and expected version.

48.5 Admin order/inventory/shipping endpoints

Method / route	Auth / permission	Request	Success	Transaction / side effects
GET /admin/orders	orders.read	status/search/cursor	200 queue	Read-only
POST /admin/orders/:reference/accept	orders.accept	expectedVersion	200 accepted order	Transaction: conditional order transition, reserve all lines, ledger, audit/outbox
POST /admin/orders/:reference/reject	orders.reject	reason, expectedVersion	200 rejected order	Transaction: terminal transition, release defensive reservation, audit/outbox
POST /admin/orders/:reference/status	module permission	target status, expectedVersion, reason	200 order	Transaction: transition map, fulfillment/payment checks, audit/outbox
POST /admin/orders/:reference/shipping	shipping.write	provider, final cost, reason, expectedVersion	200 fulfillment snapshot	Transaction: save final cost/provider; audit; customer reconfirmation policy remains configurable
POST /admin/inventory/:productId/adjust	inventory.write	variant, delta, reason, expectedVersion	200 stock summary	Transaction: conditional counter update + ledger + audit
POST /admin/orders/:reference/cod-confirmation-reminder	orders.write	optional message	202 queued	Outbox only; no state change

48.6 Service and quotation endpoints

Method / route	Auth / permission	Request	Success	Transaction / side effects
GET /services	Public	locale	200 active categories	Read-only
GET /services/:slug	Public	slug	200 category/form config	Read-only

Method / route	Auth / permission	Request	Success	Transaction / side effects
POST /services/:slug/requests	Guest/Customer	configured fields, description, contact	201 request reference/status	Transaction: create request, notification/outbox; reject attachments
GET /service-requests/:reference	Owner/Admin	reference	200 safe request state	Ownership/permission
POST /admin/service-requests/:reference/quotes	services.quote	amount, currency, note	201 quote sent	Transaction: quote + request + audit/outbox
POST /service-requests/:reference/quotation/accept	Customer ownership	quote version	200 awaiting payment	Transaction: quote accepted, request awaiting payment, payment created, audit/outbox
POST /service-requests/:reference/quotation/reject	Customer ownership	note, quote version	200 closed request	Transaction: quote rejected, request closed, Admin notification

Service request bodies are schema-derived and must reject file fields. No request creates an order/cart line.

48.7 Pre-order, return, refund endpoints

Method / route	Auth / permission	Request	Success	Transaction / side effects
POST /products/:slug/pre-orders	Guest/Customer	variant, quantity, contact	201 pre-order	Create request, Admin notification; no stock reservation
GET /pre-orders	Customer ownership/Admin	filters	200 list	Read-only
POST /admin/pre-orders/:reference/accept	preorders.write	expectedVersion, availability date optional	200 accepted	Audit/outbox; payment availability
POST /orders/:reference/returns	Customer ownership	item(s), reason, evidence metadata	201 return request	Transaction: eligibility and request creation; notification
POST /admin/returns/:reference/approve	returns.write	note, expectedVersion	200 refund initiated	Transaction: return approved, refund initiated, audit/outbox
POST /admin/returns/:reference/reject	returns.write	reason, expectedVersion	200 rejected	Transaction: terminal return state, audit/outbox
POST /admin/refunds/:refundId/complete	refunds.write	amount, method, timestamp, internal reference	200 completed refund	Transaction: refund + return completion, audit/outbox

48.8 Admin catalog/settings/report endpoints

Method / route	Auth / permission	Request	Success	Side effects
POST /admin/products	products.write	product DTO	201 product	Audit; Cloudinary metadata references validated
PATCH /admin/products/:id	products.write	partial DTO + version	200 product	Audit on price/availability/publication
POST /admin/categories	categories.write	category DTO	201 category	Audit
PATCH /admin/settings/:key	settings.write or Owner	typed value + version	200 setting	Audit; invalidates config cache
GET /admin/reports/:report	reports.read	date/status/geography filters	200 report data	Read-only aggregation
GET /admin/audit-logs	audit.read	entity/actor/date filters	200 audit list	Never exposes proof secrets

48.9 Common endpoint requirements

Every important endpoint must document in OpenAPI:

- HTTP method and route.
- Authentication and required permission.
- Path/query/body schema.
- Validation rules.
- Success envelope and status.
- Error codes.
- State and ownership preconditions.
- Transaction boundary.
- Audit event.
- Notification/outbox effect.
- Idempotency behavior where a duplicate could create a financial or operational record.

49. OpenAPI Documentation

49.1 Requirements

- Generate OpenAPI 3 documentation from Zod schemas or a synchronized typed source.
- Serve Swagger UI only in development or behind Admin protection in production.
- Publish a versioned JSON document with /api/v1 paths.
- Include bearer auth and refresh-cookie security schemes.
- Include all error codes and common envelopes.
- Include Arabic/English examples where localized fields matter.
- Export a Postman collection from the same endpoint contract.

49.2 Documentation checks

CI must fail when a route is registered but absent from OpenAPI, when an enum differs between schema and documentation, or when an endpoint has no error response definition.

50. Security Architecture

50.1 Controls

- Helmet with secure headers.
- Strict CORS, never * in production.
- Zod validation and allowlisted query operators.
- Mongoose strict schemas and sanitized filters to prevent NoSQL injection.
- Argon2id password hashing.
- Hashed refresh/reset/guest tokens.
- HttpOnly/Secure refresh cookie.
- Access-token expiry and session revocation.
- Rate limiting by IP plus account/email-sensitive buckets.
- Request body and upload metadata limits.
- Safe error responses with request IDs.
- Permission and record-ownership checks.
- Private Cloudinary proofs and short-lived signed access.
- Audit of critical mutations.
- No sensitive data in logs, URLs, Socket.IO payloads, or WhatsApp prefills.

50.2 CSRF

The access token is sent in an Authorization header; the refresh token is cookie-based. Configure SameSite appropriately for the same-origin deployment. If cross-site cookie usage is enabled later, add a CSRF token mechanism for refresh/logout and any cookie-authenticated mutation. Do not treat CORS as CSRF protection.

50.3 XSS and injection

The API stores text as data and does not render HTML. If rich content is later approved, sanitize with a strict allowlist before storage and response. Never pass user input into MongoDB operators, shell commands, Cloudinary transformations, email templates, or WhatsApp URLs without encoding.

50.4 Data exposure

Customer endpoints never expose other customers' records, internal audit detail, stock counters, signed proof URLs, passwords, token hashes, or Admin notes unless explicitly allowed by the source contract.

51. Rate Limiting

Use layered limits with environment configuration:

Bucket	Applies to
Public IP	Catalog, search, content
Auth IP	Login, registration, refresh
Account/email	Login attempts, reset requests, verification
Guest order	Order creation and guest order lookup
Proof upload	Payment proof metadata submission
Admin mutation	Sensitive operational mutations
Socket handshake	Connection attempts

Return 429 `RATE_LIMITED` with a retry hint where safe. Do not reveal whether an email/account exists.

52. CORS

- Production origins are explicit environment values.
 - Development may allow configured localhost origins.
 - Credentials are enabled only when the refresh cookie is used.
 - Allowed methods and headers are allowlisted.
 - Preflight responses are cached for a bounded period.
 - Never use `origin: "*" with credentials or as a production default.`
-

53. Logging & Observability

53.1 Pino fields

Every request log includes `requestId`, `timestamp`, `method`, `path`, `status`, `duration`, `user ID` where safe, `role`, and `error code`. Hash IP/device identifiers when retained. Log domain events at the service boundary, not every database query.

53.2 Do not log

Passwords, `access/refresh/reset/guest` tokens, cookie values, Cloudinary signatures, payment proof URLs, full addresses, payment details, or raw customer notes unless explicitly redacted.

53.3 Health endpoints

- `GET /health/live`: process is alive; no database dependency.

- GET /health/ready: database and required configuration are ready; do not expose secrets.

Track:

- request latency/error rate,
- database latency and transaction aborts,
- outbox backlog and terminal failures,
- email/socket delivery failures,
- rate-limit events,
- inventory conflicts,
- Cloudinary upload metadata failures,
- cron duration/failures.

54. Environment Configuration

54.1 .env.example

```

NODE_ENV=development
PORT=3000
API_BASE_PATH=/api/v1
PUBLIC_APP_ORIGIN=http://localhost:4200
ALLOWED_ORIGINS=http://localhost:4200

MONGODB_URI=
MONGODB_DB_NAME=al_azhari_library

JWT_ACCESS_SECRET=
JWT_ACCESS_TTL=15m
JWT_ISSUER=al-azhari-library
JWT_AUDIENCE=al-azhari-web
REFRESH_COOKIE_NAME=al_azhari_refresh
REFRESH_COOKIE_SECURE=false
REFRESH_COOKIE_SAME_SITE=lax

ARGON2_MEMORY_COST=
ARGON2_TIME_COST=
ARGON2_PARALLELISM=

CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
CLOUDINARY_PAYMENT_PROOF_FOLDER=al-azhari/payment-proofs
CLOUDINARY_PRODUCT_FOLDER=al-azhari/products

SMTP_HOST=
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=
SMTP_PASSWORD=
EMAIL_FROM=

WHATSAPP_PHONE=
SOCKET_PATH=/socket.io
RATE_LIMIT_PUBLIC_PER_MINUTE=
RATE_LIMIT_AUTH_PER_MINUTE=
OUTBOX_POLL_INTERVAL_MS=
OUTBOX_MAX_ATTEMPTS=

```

Blank values are intentional placeholders, not credentials. Secrets are provided through Replit/Hostinger environment configuration and never committed.

54.2 Startup validation

Zod validates environment by environment:

- Development may use local non-secure cookies.
- Test uses an isolated MongoDB URI and fake adapters.
- Production requires non-empty secrets, explicit origins, secure cookies, and Cloudinary/SMTP configuration if those features are enabled.

Fail startup on missing required production configuration. Never silently fall back to insecure defaults.

55. Testing Strategy

55.1 Tools

- Jest for unit and integration tests.
- Supertest for HTTP API tests.
- MongoDB Memory Server for isolated database tests.
- Fake Cloudinary/email/socket adapters for deterministic tests.
- Concurrency tests against a replica-capable test database where transaction behavior must be verified.

55.2 Test layers

Layer	Minimum coverage
Unit	Money, normalization, state maps, coupon calculation, shipping rule match, permission checks
Schema	Required fields, enums, unique indexes, strict unknown-field rejection, TTL indexes
API	Status codes, envelopes, Zod errors, auth, ownership, pagination
Workflow	Registration, reset, guest checkout, registered checkout, cart merge, payment proof, COD, services, returns/refunds
Concurrency	Last-stock acceptance, duplicate checkout, duplicate proof, stale Admin version, duplicate refund
Security	No token/password leakage, NoSQL injection rejection, CORS, rate limits, signed media authorization
Outbox	Atomic creation, lease recovery, retry/backoff, dedupe, terminal failure
Deployment smoke	Health, database connection, startup validation, migration/seed

55.3 Major-module acceptance criteria

Auth: duplicate email/phone is rejected; login by email and phone works; reset tokens are hashed/one-time; revoked refresh tokens fail.

Catalog: unpublished/future categories are hidden; variants require selection; null descriptions do not block publish; search works with partial metadata.

Cart/checkout: guest and registered flows work; current price/availability are revalidated; no service enters cart; duplicate submit returns one order.

Inventory: order creation does not deduct; acceptance reserves atomically; rejection/cancellation release; delivery/pickup deduct exactly once.

Payments: non-COD requires proof; COD does not; Admin actions are audited; proof URLs stay private.

Services: request/quote/payment are separate; no attachment field is accepted; quote acceptance is required before payment.

Returns/refunds: only completed eligible orders can request; refund is distinct and manually recorded; duplicate completion is safe.

Notifications: lifecycle records persist even when socket/email fails; dedupe prevents duplicate user notifications.

RBAC: Admin cannot exceed assigned permission; Owner can operate all baseline modules; customer cannot read another customer's records.

55.4 Example test

```
it("does not oversell the last unit when two admins accept concurrently", async () => {
  const [first, second] = await Promise.allSettled([
    acceptOrder(orderA.reference, adminContext),
    acceptOrder(orderB.reference, adminContext),
  ]);

  expect([first, second].filter((r) => r.status === "fulfilled")).toHaveLength(1);
  expect([first, second].filter((r) => r.status === "rejected")).toHaveLength(1);
  expect(await getAvailableStock(productId)).toBe(0);
});
```

56. Deployment Architecture

56.1 Runtime

Build TypeScript to `dist/`. Run one Node.js server process that serves the API and attaches Socket.IO. The Angular application may be served by the same origin or a configured frontend host; the API origin and CORS must remain explicit.

56.2 Release sequence

1. Install dependencies with lockfile.
2. Validate production environment.
3. Run versioned migrations/index checks.
4. Run seed only for safe system roles/settings and never overwrite business data.
5. Start the new process.
6. Verify `/health/live` and `/health/ready`.
7. Run smoke requests.
8. Monitor outbox, database, and error metrics.

Never run destructive migrations automatically as part of startup.

57. Hostinger Deployment

1. Configure a Hostinger Node.js application with the supported Node runtime.

2. Set the start command to the compiled server, for example `node dist/server.js`.
 3. Provide environment variables through Hostinger's secure configuration.
 4. Configure the frontend/API same-origin strategy or explicit production origin.
 5. Enable HTTPS and secure refresh cookies.
 6. Ensure websocket upgrade support is enabled for Socket.IO; if unavailable, the application still works through persisted refresh state.
 7. Ensure the process can write only to permitted temporary locations; media is not stored locally.
 8. Configure a single active worker instance or a lease-safe worker if the host runs multiple instances.
 9. Run migrations as a controlled deployment step.
 10. Keep logs available to operators without logging secrets.
-

58. Database Deployment

- Use MongoDB Atlas replica set/tier with transactions.
 - Create a dedicated least-privilege application database user.
 - Restrict network access to approved Hostinger egress where practical.
 - Enable TLS.
 - Apply collection validators/indexes through a versioned migration runner.
 - Monitor connections, replication lag, storage, slow queries, transaction aborts, and index usage.
 - Keep application and test databases separate.
 - Never run test cleanup against production URI.
-

59. Backup & Recovery Considerations

- Enable Atlas backup/point-in-time recovery according to the chosen Atlas tier.
 - Document recovery owner, RPO, RTO, and retention with the owner; they are not specified by the product sources.
 - Test restore into a separate database before production launch and periodically thereafter.
 - Store no irreplaceable payment proof only in local application storage; Cloudinary retention must align with database backup policy.
 - Back up configuration needed to reconstruct settings and shipping rules without exporting secrets.
 - After restore, verify unique indexes, TTL indexes, outbox states, and audit immutability.
-

60. Performance Considerations

- Use lean projections for public lists and Admin queues.
- Paginate every unbounded list.
- Project only safe fields.
- Avoid `populate()` in hot transaction paths.

- Keep transactions short and never call Cloudinary, SMTP, or Socket.IO while a MongoDB transaction is open.
- Cache read-only settings with version invalidation after Admin update.
- Use stable product/category indexes and measure query plans.
- Bound `limit` values server-side.
- Keep status histories bounded; use audit logs for detail.
- Process outbox work in batches with a bounded batch size.
- Do not introduce read models or daily aggregates until query evidence shows they are needed.
- Surface stale cart/order conflicts rather than hiding them.

61. Implementation Phases

Phase	Objective and modules	Database/API work	Tests / Definition of Done
0. Foundation	TypeScript, Express, lint, strict config	App bootstrap, request ID, error envelope	Server boots; health routes
1. Configuration & DB	config, database	Env schema, Mongoose connection, migrations, validators, indexes	Isolated DB connects; migrations repeat safely
2. Common infrastructure	errors, middleware, HTTP	Helmet, CORS, Pino, Zod, rate limits	Middleware and error tests
3. Authentication	auth, users, sessions	Users, sessions, auth tokens, roles	Register/login/refresh/logout/reset/verification
4. Users & addresses	users, addresses	Profile/address CRUD and ownership	Ownership/security tests
5. Catalog	products, categories, content	Products, variants, categories, media metadata, public reads	Catalog/search/publication tests
6. Cart	carts	Guest/user persistence and merge	Cart version/merge tests
7. Inventory	inventory	Reservations, ledger, conditional updates	Race/transaction tests
8. Checkout & orders	orders, shipping	Shipping rules, totals, order transaction, guest token	Checkout/idempotency tests
9. Payments	payments, proof, Cloudinary	Manual proof lifecycle and Admin review	Proof authorization/state tests
10. Shipping & coupons	shipping, coupons	Estimate/final cost, coupon calculation/redemption	Rule precedence/coupon tests
11. Services	services, quotations	Configurable forms, request/quote lifecycle	No-attachment/service-boundary tests
12. Returns & refunds	returns, refunds	Return eligibility, manual refund records	Transition/idempotency tests
13. Notifications/realtime	notifications, socket, email	Persisted notification, Socket.IO rooms, email adapter	Delivery failure fallback
14. Outbox/jobs	outbox, jobs	Worker, retry, lease recovery, cron	Crash/retry/dedupe tests

Phase	Objective and modules	Database/API work	Tests / Definition of Done
15. Reports/audit	reports, audit	Aggregations, immutable audit queries	Report permission and redaction
16. Full testing	all modules	Integration and concurrency suite	Acceptance criteria pass
17. Security hardening	all	Threat review, secret audit, limits, media review	Security test suite pass
18. Deployment	Hostinger/Atlas	Production config, migration, restore smoke	Live health, rollback plan, monitored release

Each phase's files/components are the module folders in Section 8 plus tests under `test/unit`, `test/integration`, `test/api`, and `test/workflows`. A phase is not done when only routes compile; its state transitions, authorization, side effects, tests, and operational behavior must be complete.

62. Definition of Done

A backend feature is done only when:

- Its domain boundary and source requirement IDs are recorded.
- Request schemas, response DTOs, and error codes exist.
- Authentication, permission, ownership, and redaction are tested.
- State transitions are explicit and invalid transitions fail.
- Transactions/idempotency are applied where required.
- Audit, notification, and outbox effects are implemented where required.
- Indexes and migration changes are versioned.
- Open decisions are not silently hard-coded.
- Unit, integration, API, and relevant concurrency/security tests pass.
- OpenAPI and Postman documentation match the route.
- Logs do not expose secrets or unnecessary PII.
- Health/readiness and deployment behavior are verified.

63. Acceptance Criteria

Product/order

- A guest can browse, add products/variants, choose pickup/delivery, see a shipping estimate or free pickup, choose an enabled payment method, and submit an order.
- A registered customer can perform the same flow and later view their own history.
- A new order is `pending_review`; no stock is deducted.
- Admin acceptance reserves all eligible stock atomically.
- COD requires customer confirmation after Admin acceptance.

- Non-COD payment requires screenshot proof and Admin review.
- Admin rejection/request-new-proof is visible without exposing internal audit detail.
- Fulfillment reaches the correct pickup or delivery states.
- Stock is finally deducted exactly once at pickup/delivery.

Services

- Services are discoverable and requestable but never cart items.
- Required fields are driven by approved configuration.
- No service attachment upload is accepted.
- Admin can quote; customer can accept/reject.
- Payment is unavailable before quote acceptance.

Operational/security

- Admin capabilities follow RBAC.
- Owner can perform all baseline Admin operations.
- Audit records exist for critical mutations.
- Socket/email failure does not lose the state change.
- Refresh/session revocation works.
- API errors are stable and safe.
- Production configuration refuses insecure or missing secrets.

64. Risk Register

Risk	Impact	Mitigation / owner
Payment-method discrepancy	Checkout policy may be wrong	Resolve OD-20; configure methods; Owner
Shipping rate logic not approved	Incorrect totals or serviceability	Use shippingRules; block unmatched areas; Owner/Operations
Final shipping cost binding unclear	Customer disputes	Keep estimate/final separate; resolve OD-04
Product descriptions incomplete	Lower conversion/trust	Allow publish without description; Admin content workflow
Arabic search quality	Missed catalog results	Normalized field queries; validate text behavior before enabling
Last-stock concurrency	Overselling	Conditional reservation + transactions + conflict response
Manual proof privacy	Sensitive media exposure	Private Cloudinary assets, signed URLs, redaction
Service fields undefined	Incorrect requests	Configurable form schema; no invented fields
COD confirmation abandonment	Orders remain pending	Do not auto-cancel until timeout policy is approved
Outbox worker crash	Delayed notifications	Lease recovery, idempotent events, operational backlog
Hostinger websocket limitations	Live updates unavailable	Persisted notifications and refresh fallback
Restore not tested	Long outage/data loss	Atlas backup and restore drill

Risk	Impact	Mitigation / owner
Coupon governance undefined	Discount leakage	Centralized policy config; block unspecified rules
Retention undefined	Privacy/operational conflict	Resolve retention owner and policy before cleanup

65. Open / Not Specified Items

Only the following items remain open after reviewing the supplied sources. They must be resolved by the named owner/stakeholder before dependent production behavior is hard-coded.

ID	What is missing	Why it matters	Defining document	Owner
OD-01	Final website phone number	Contact/footer and WhatsApp support	Master Brief / settings	Owner
OD-02	Exact email-verification gating scope	Determines which account actions are blocked	PRD AUTH-005	Owner + Product
OD-03	Shipping rate source/logic	Required to calculate delivery estimates	PRD SHIP-007	Owner + Operations
OD-04	When final shipping cost binds and whether reconfirmation is required	Prevents cost disputes	PRD SHIP-006	Owner + Product
OD-05	Delivery time ranges	Avoids unsupported promises	PRD SHIP-008	Operations
OD-06	Excluded areas/serviceability policy	Controls delivery acceptance	PRD SHIP-009	Operations
OD-07	Reservation expiry policy	Determines whether/when reserved stock is released	PRD/DB plan	Owner + Operations
OD-08	Fairness rule for competing last units	Determines queue/allocation behavior	PRD INV-007	Owner
OD-09	Low-stock/reorder thresholds	Controls alerts and reports	PRD INV-008	Owner
OD-10	Whether pre-order price is honored after catalog changes	Determines later payment/fulfillment amount	PRD PRE-007	Owner
OD-11	Allocation order for multiple pre-orders	Determines who receives newly available stock	PRD PRE-007 / DB plan	Owner
OD-12	Required fields per service type	Defines request schemas and UI	PRD SRV-006	Owner + Service staff
OD-13	Service turnaround commitments	Controls customer messaging	PRD SRV-007	Service staff
OD-14	Whether COD applies to accepted service quotations	Controls service payment methods	PRD QUOTE-005	Owner
OD-15	Category-specific return periods/rules	Controls eligibility	PRD RET-004	Owner
OD-16	Refund processing SLA	Controls customer expectation/reporting	DB plan	Owner

ID	What is missing	Why it matters	Defining document	Owner
OD-17	Coupon stacking, limits, expiry, minimum, customer restrictions	Prevents discount abuse	PRD COUP-004	Owner
OD-18	KPI targets	Prevents invented business targets	PRD REP-002	Owner
OD-19	Operational notification/email policy details	Determines recipients/templates	PRD NOT-002/004	Owner + Operations
OD-20	Final payment-method list and cash-at-library selection	Resolves PRD/Brief conflict	PRD CHK-007 / Master Brief	Owner
OD-21	Partial acceptance behavior	Determines mixed-stock order policy	PRD/DB plan	Owner
OD-22	Payment-proof retention period and deletion policy	Controls privacy and Cloudinary cleanup	Security/Operations policy	Owner
OD-23	Backup RPO/RTO and retention	Determines recovery configuration	Operations runbook	Owner + Hosting
OD-24	Exact post-acceptance non-financial order edits	Controls address/contact changes	PRD ORD-009	Owner + Operations

Until an item is resolved, the implementation must use a safe neutral behavior: configuration, explicit pending state, or rejection with a recoverable explanation. No numeric price, deadline, threshold, promise, or policy is invented.

66. Final Technical Decisions

All decisions below are final for this implementation plan. They are architecture decisions, not a reopening of the business open items in Section 65.

#	Area	Decision	Status
1	Architecture	Use a modular monolith	FINAL
2	Runtime	Use Node.js	FINAL
3	Framework	Use Express.js	FINAL
4	Language	Use TypeScript	FINAL
5	Database	Use MongoDB Atlas	FINAL
6	ODM	Use Mongoose	FINAL
7	API	Use REST	FINAL
8	API versioning	Use <code>/api/v1</code>	FINAL
9	Frontend boundary	Consume from Angular	FINAL
10	Hosting	Deploy Node.js API on Hostinger	FINAL
11	Access auth	Use JWT access tokens	FINAL
12	Refresh auth	Use opaque refresh tokens in HttpOnly/Secure cookies	FINAL

#	Area	Decision	Status
13	Session persistence	Store hashed refresh/session records in MongoDB	FINAL
14	Session control	Support session revocation and rotation	FINAL
15	Password hashing	Use Argon2id	FINAL
16	Password reset	Use opaque random hashed reset tokens, not JWT	FINAL
17	Guest checkout	Support guest checkout with a guest order token	FINAL
18	Account merge	Merge guest records only after email verification	FINAL
19	Authorization	Use RBAC with granular permission keys	FINAL
20	Admin creation	Owner only may create Admin users	FINAL
21	HTTP security	Use Helmet and secure HTTP defaults	FINAL
22	Throttling	Use public, IP, account, and sensitive-endpoint rate limits	FINAL
23	CORS	Use environment-configured explicit origins; never production *	FINAL
24	Response format	Use a standardized success/error envelope	FINAL
25	Validation	Use Zod at all request boundaries	FINAL
26	Errors	Use centralized typed AppError handling	FINAL
27	Logging	Use Pino structured logging	FINAL
28	API docs	Maintain OpenAPI/Swagger and Postman artifacts	FINAL
29	Product media	Use Cloudinary with MongoDB metadata	FINAL
30	Upload direction	Upload media directly from frontend through constrained signed configuration	FINAL
31	Email	Use an SMTP/Gmail/private-mail adapter, not a mandatory new provider	FINAL
32	Realtime	Use Socket.IO	FINAL
33	Socket auth	Authenticate Socket.IO with the access token	FINAL
34	Socket rooms	Use user/order/service rooms with ownership checks	FINAL
35	Notifications	Persist MongoDB notification records	FINAL
36	Delivery fallback	Persisted state remains authoritative when realtime fails	FINAL
37	Background work	Use MongoDB Outbox Pattern	FINAL
38	Worker	Use an internal worker in the same Node.js process	FINAL
39	Queue dependency	Do not require BullMQ or Redis for MVP	FINAL
40	Scheduling	Use Node cron for scheduled work	FINAL
41	Cart	Persist guest and registered carts in MongoDB	FINAL
42	Checkout idempotency	Use internal idempotency; frontend need not supply a mandatory special header	FINAL
43	Payments	Use manual payment workflow only	FINAL
44	Payment proofs	Store proof metadata in MongoDB and media in Cloudinary	FINAL
45	Payment review	Support Admin confirm, reject, request-new-proof, and note actions	FINAL

#	Area	Decision	Status
46	Inventory	Reserve at order acceptance and deduct at delivered/picked-up	FINAL
47	Search	Use normalized fields, MongoDB indexes, filters, sorting, and pagination initially	FINAL
48	Shipping	Use configurable shippingRules, free pickup, estimated delivery cost, and Admin final adjustment	FINAL
49	Services	Keep student services separate from cart/orders and use request → quote → decision → payment → fulfillment	FINAL
50	Testing	Use Jest, Supertest, MongoDB integration tests, and MongoDB Memory Server	FINAL

Final consistency check

- PRD requirements are represented across the module and API sections.
- The MongoDB collection intent, snapshots, indexes, state separation, and transaction requirements are preserved.
- The locked Node.js/Express/TypeScript/MongoDB/Mongoose/Hostinger architecture is used.
- Order, payment, and fulfillment states are separate.
- Guest checkout and guest access are fully represented.
- Manual proof review and COD confirmation are explicit.
- Student services cannot enter the product cart.
- Notifications are persisted; Socket.IO/email are delivery layers.
- Outbox work is post-commit, retryable, and idempotent.
- Cloudinary is used for approved media only; service attachments remain prohibited.
- RBAC and ownership checks are required.
- Validation is Zod-based and errors are centralized.
- Testing covers workflows, concurrency, security, transactions, inventory, payments, services, returns, refunds, notifications, and outbox behavior.
- No business rate, SLA, threshold, retention period, or unresolved policy was invented.

Requirement traceability matrix

This matrix is the implementation handoff index. A feature is not considered covered because its requirement is mentioned in prose; the listed module, endpoint, persistence boundary, and test family must exist.

Requirement group	Backend module(s)	Primary API surface	Persistence / transaction	Minimum verification
AUTH-001–012 Authentication & accounts	auth, users, addresses	/auth/*, /me, /addresses	users, sessions, authTokens, addresses; session transaction for rotation/revocation	Auth API, token security, ownership
CAT-001–009, VAR-001–004 Catalog & variants	products, categories, content	/products, /categories, /admin/products	products, categories, contentModules; single-document writes plus audit	Publication, variant, price, media
SEARCH-001–005 Search & filters	products	/search, /products	Normalized product fields and indexes	Arabic/Latin query and filter tests

Requirement group	Backend module(s)	Primary API surface	Persistence / transaction	Minimum verification
PDP-001-005 Product detail	products	/products/:slug	Public projection of products	Availability and variant-selection tests
CART-001-005 Cart	carts, products	/cart/*	carts; optimistic version checks	Guest persistence, merge, stale version
CHK-001-012 Checkout	carts, orders, payments, shipping, coupons	/checkout/shipping-estimate, POST /orders	Order/payment/coupon/outbox transaction	Guest, registered, changed-price, proof
ORD-001-009 Product orders	orders, inventory, notifications	/orders/*, /admin/orders/*	orders, snapshots, status history; transition transactions	State machine and duplicate mutation
PAY-001-008 Manual payments	payments, paymentProofs, integrations/cloudinary	/payment-proofs/*, /admin/payments/*	Payment/proof/order/audit/outbox transactions	Privacy, review, re-upload
SHIP-001-009, PICK-001-004 Fulfillment	shipping, orders	/checkout/shipping-estimate, Admin shipping/status routes	shippingRules, order fulfillment snapshot	Rule precedence and pickup/delivery
INV-001-008 Inventory	inventory, products, orders	Admin inventory/order acceptance routes	Reservations, ledger, conditional stock transactions	Overselling and final deduction
PRE-001-007 Pre-orders	preorders, products, payments	/products/:slug/pre-orders, Admin pre-order routes	preOrders; no reservation before availability	Eligibility, price snapshot, handoff
SRV-001-008 Student services	services, notifications	/services, /service-requests/*	serviceCategories, serviceRequests; request transaction	No cart entry, no attachment field
QUOTE-001-005 Quotations	quotations, services, payments	Admin quote routes and customer decision routes	Quote/request/payment transaction on acceptance	Version conflict and payment gate
RET-001-004, REF-001-003 Returns/refunds	returns, refunds, inventory	Customer return and Admin refund routes	returnRequests, refunds; approval/completion transactions	Eligibility, duplicate completion
NOT-001-004, RT-001-003 Delivery	notifications, realtime/socket, integrations/email	Notification routes, Socket.IO events	notifications, outboxEvents	Delivery failure fallback and dedupe
COUP-001-005 Coupons	coupons, orders	Checkout coupon field, Admin coupon routes	coupons, couponRedemptions; order transaction	Scope, idempotency, open policy
ADM-001-003, RBAC-001-004 Operations/RBAC	all Admin modules, roles, audit	/admin/*	roles, auditLogs; critical mutations transactional	Permission matrix and cross-tenant ownership

Requirement group	Backend module(s)	Primary API surface	Persistence / transaction	Minimum verification
CMS-001–004 Content	content, products	/content/home, Admin content routes	contentModules, product descriptions	Seasonal layer cannot mutate taxonomy
REP-001–002 Reports	reports	/admin/reports/*	Read-only aggregations	Dates, geography, payment, demand
WA-001–003 WhatsApp	integrations/whatsapp	Safe URL response/action data	No authoritative record mutation	Encoding and secret-redaction tests

Critical workflow implementation checklist

The following checklist is the minimum implementation contract for the backend team. Each row must be linked to a test file and a migration/model change before the phase is closed.

Workflow	Trigger and preconditions	Database changes / transaction	External effects	Recovery requirement
Registration	Guest submits valid unique name, phone, email, password	Create user, hashed verification token, notification/outbox; transaction	Verification email	Duplicate identity returns conflict without partial user
Login	Valid email/phone and password; active account	Create/rotate session; update login metadata	Access token and refresh cookie	Repeated invalid attempts are throttled
Refresh/logout/revoke	Valid refresh cookie or authenticated session	Rotate session hash or mark revoked; version bump for global revoke	None or audit	Replayed refresh token fails
Email verification	Valid unconsumed token	Consume token and set emailVerifiedAt	Confirmation notification	Expired token is safe and retryable through a new request
Password reset	Neutral reset request; valid one-time token for reset	Replace hash, consume token, revoke sessions	Reset email	Unknown email does not reveal existence
Guest checkout	Eligible cart, valid contact, fulfillment, payment, coupon	Create order/payment/token/cart update/outbox atomically	Confirmation email/in-app notification	Idempotent replay returns original order
Registered checkout	Same as guest plus account ownership	Same order transaction linked to user	Account notification	Cart/order ownership remains enforced
Cart merge	Guest session logs into customer account	Merge compatible lines with version check	None	Conflicting/unavailable lines returned explicitly
Payment proof submission	Owner of accepted payment; valid Cloudinary metadata	Create proof and update payment/order/audit/outbox atomically	Admin/customer notifications	Duplicate key cannot create a second active submission
Payment review	Authorized Admin; expected payment/proof version	Update proof/payment/order, audit, notification/outbox atomically	Socket/email delivery	Failed delivery does not undo review

Workflow	Trigger and preconditions	Database changes / transaction	External effects	Recovery requirement
COD confirmation	Accepted COD order in customer-confirmation state	Transition order to confirmed with version predicate	Admin/customer notification	Repeated confirmation is idempotent
Inventory reservation	Admin accepts pending order; all lines available	Conditional stock reservation, reservation rows, ledger, order transition atomically	Queue/status notification	Any failed line aborts the whole acceptance
Reservation release	Rejection/cancellation or approved release policy	Decrement reserved, mark reservation released, ledger, state change	Notification where applicable	Repeated release does not decrement twice
Fulfillment deduction	Order reaches delivered/picked-up; reservation active	Decrement total/reserved, consume reservation, advance order, ledger atomically	Completion notification	Repeated fulfillment returns existing completion
Service request	Configured service category; no attachment fields	Create request and Admin notification transaction	Admin/customer notification	Invalid dynamic field set is rejected without record
Quotation	Admin reviews request and supplies final amount	Create versioned quote, update request, audit/outbox	Quote notification	Stale quote version cannot overwrite newer quote
Quote acceptance/rejection	Customer owns sent quote	Acceptance creates payment and updates request atomically; rejection closes request	Customer/Admin notification	Duplicate decision returns existing decision
Pre-order	Product is out of stock and eligible	Create demand snapshot; no inventory reservation	Admin notification	Invalid/deactivated product is rejected
Return/refund	Completed eligible order; authorized Admin for decision	Return approval/refund initiation, then refund completion as separate transactions	Customer/Admin notification	Duplicate approval/completion is safe
Outbox delivery	Pending/expired processing event	Claim lease, update attempt/status	Email/socket adapter	Crash recovery reclaims stale processing events

Refinement exit criteria

- Every requirement group above has at least one route or explicit reason it is persistence-only.
- Every critical workflow has a transaction boundary, audit/notification behavior, and recovery behavior.
- Open decisions in Section 65 remain configuration or blocked behavior.
- The PDF is regenerated from this exact Markdown source, so the editable and primary deliverables cannot drift.